

BLOCKCHAIN SHOPPING SYSTEM

Offline / Local Network Full Implementation Handbook

Hyperledger Fabric + TypeScript + Node.js + PostgreSQL + Local Web/POS

200+ page engineering guide for an Internet-independent permissioned blockchain shopping platform

Reference baseline: Examples use Hyperledger Fabric 2.5.x LTS as the conservative production baseline for the August 2026 documentation set. Fabric 3.x concepts are covered separately in the advanced/upgrade section. All normal runtime operations are designed to work with the WAN/Internet disconnected.

Prepared: August 2026

Architecture, implementation, security, testing, operations, and deployment

How to Use This Handbook

This handbook is written as a full implementation reference. Follow Parts I-III before building the network; they define the trust model, data boundaries, and offline infrastructure assumptions. Parts IV-VI build the Fabric network, smart contracts, and API integration. Parts VII-VIII implement user workflows. Parts IX-XI harden, test, and operate the system. Part XII covers performance and advanced architecture. Part XIII provides reproducible implementation labs. The appendices collect reference code, schemas, checklists, and official sources.

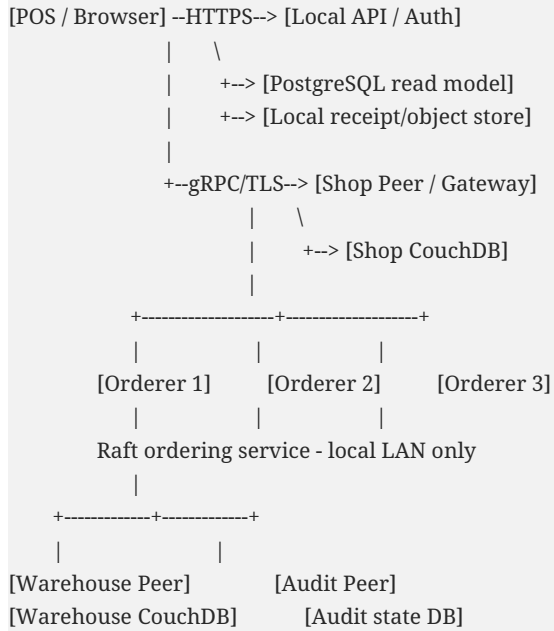
Important architecture warning: If one person controls every peer, orderer, CA, application server, and private key, a blockchain cannot create independent organizational trust by itself. It can still provide an append-only, tamper-evident transaction history, but a simpler signed append-only database may be more appropriate. This handbook deliberately separates ShopOrg, WarehouseOrg, and AuditOrg administrative responsibilities to make the blockchain assurance meaningful.

System Outcome

- A customer/cashier can browse products, check out, pay locally, receive a receipt, and perform permitted returns without Internet access.
- Inventory movements and order/payment/refund facts are committed to a permissioned ledger with organization identities and endorsement policies.
- Customer PII, passwords, images, and operational search indexes remain off-chain and manageable under local retention rules.
- The platform continues normal operation on a private LAN and has documented offline patching, backup, restore, certificate rotation, and disaster-recovery procedures.

Executive Reference Architecture

LAN (no Internet required)



Item	Implementation Guidance
Ledger layer	Hyperledger Fabric peers, Raft orderers, channel, chaincode, MSP identities and policies.
Application layer	Node.js/TypeScript REST API using Fabric Gateway; event projection worker; local authentication.
Operational data	PostgreSQL read/projection database; local receipt/document store; logs and monitoring.
Clients	Browser storefront, cashier/POS, warehouse console, manager/admin, audit dashboard.
Network	Private Ethernet/VLANs, local DNS, local time service, TLS, no required WAN path.

Detailed Contents

Part I - Architecture, Trust Model, and Blockchain Decisions

1. Why Use Blockchain in a Local Shopping System
2. Reference Architecture for an Offline LAN Store
3. Trust Boundaries and Administrative Domains
4. What Goes On-Chain vs Off-Chain
5. Permissioned Blockchain Concepts for Beginners
6. Fabric 2.5 LTS Baseline and Upgrade Strategy
7. Transaction Finality and Why It Matters for Retail
8. Reference Non-Functional Requirements

Part II - Shopping Domain and Ledger Data Model

9. Shopping Actors, Roles, and Responsibilities
10. Product Catalog Model
11. Inventory and Stock Movement Model
12. Cart and Reservation Model
13. Order Header and Order Line Model
14. Payment Recording Without Public Cryptocurrency
15. Refund and Return Model
16. Discounts, Coupons, and Promotions
17. Receipts, Documents, and Hash Anchoring
18. Data Retention, Deletion, and Privacy Boundaries

Part III - Offline LAN Infrastructure

19. Physical LAN Topology
20. Small Single-Host Lab vs Multi-Server Production
21. Local DNS and Hostname Strategy
22. Local Time Synchronization
23. Offline Software Supply Chain
24. Docker and Container Runtime in an Air-Gapped Environment
25. Offline NPM and Application Dependency Management
26. Local Certificate and Secret Storage

27. Storage Layout and Filesystem Design

28. Local Monitoring and Administration Network

Part IV - Hyperledger Fabric Network Deployment

29. Fabric Installation Bundle for Offline Use

30. Organization Design: ShopOrg, WarehouseOrg, AuditOrg, OrdererOrg

31. Fabric CA Server Deployment

32. Registering and Enrolling Identities

33. MSP Directory Construction

34. TLS Certificates for Internal Services

35. Deploying the Raft Ordering Service

36. Deploying Shop, Warehouse, and Audit Peers

37. CouchDB State Database Design

38. Channel Configuration

39. Creating and Joining the Shopping Channel

40. Anchor Peers, Discovery, and Gateway Connectivity

Part V - Smart Contracts and Chaincode

41. Chaincode Project Structure in TypeScript

42. Deterministic Smart Contract Design

43. Composite Key Strategy

44. Product Contract Implementation

45. Inventory Contract Implementation

46. Order Contract Implementation

47. Payment Record Contract

48. Return and Refund Contract

49. Role and Attribute Authorization in Chaincode

50. Business Rule Validation

51. Idempotency at Ledger Level

52. MVCC and Concurrent Checkout

53. Chaincode Events

54. Rich Queries and CouchDB Indexes

55. Private Data Collections

Part VI - Backend API and Fabric Gateway Integration

56. Backend Service Architecture

57. Node.js/TypeScript Project Bootstrap

58. Fabric Gateway Connection Helper

59. Identity Mapping for Staff and Services

60. Authentication and Session Management

61. Product REST API

62. Inventory REST API

63. Order and Checkout REST API

64. Payment and Refund REST API

65. Chaincode Event Listener and Projection Worker

66. PostgreSQL Read Model

67. API Error Model, Correlation IDs, and Logging

Part VII - Local Web/POS User Interface

68. Frontend Architecture for LAN-Only Operation

69. Login and Role-Aware Navigation

70. Catalog and Product Search

71. Cart and Checkout UX

72. Cashier/POS Workflow

73. Warehouse and Inventory Screens

74. Audit and Verification Dashboard

Part VIII - End-to-End Business Flows

75. End-to-End Product Creation Flow

76. Stock Receiving Flow

77. Customer Checkout Flow

78. Concurrent Checkout Conflict Recovery

79. Return and Refund Flow

80. Order Cancellation and Reservation Release

81. Receipt Verification Flow

82. Daily Reconciliation and Closing

Part IX - Security, Privacy, and Abuse Resistance

83. Threat Model for an Offline Blockchain Store

84. Network Segmentation and Firewall Policy

85. Private Key Protection

86. Certificate Renewal and Revocation

87. API Authorization and Least Privilege

88. PII Minimization and Pseudonymization

89. Secure Offline Update Process

90. Incident Response and Kill Switches

Part X - Testing and Quality Engineering

91. Chaincode Unit Testing Strategy

92. Fabric Integration Test Environment

93. Checkout and Inventory Concurrency Tests

94. Negative and Security Test Cases

95. Network Failure and Quorum Tests

96. Backup and Restore Tests

97. User Acceptance and Store Operations Testing

Part XI - Operations, Backup, and Disaster Recovery

98. Daily Startup and Shutdown Procedures

99. Health Monitoring and Local Alerting

100. Backup Architecture

101. Ledger Snapshots and Peer Recovery

102. PostgreSQL Projection Rebuild

103. Chaincode Upgrade and Migration

104. Certificate Rotation Operations

Part XII - Performance, Scaling, and Advanced Designs

105. Performance Measurement for Checkout Workloads

106. Block Cutting and Ordering Tradeoffs

107. Peer and CouchDB Resource Sizing

108. State-Based Endorsement for High-Risk Records

109. Multiple Stores and Branches

110. Supplier Organization Integration

111. Upgrade Path to Fabric 3.x and SmartBFT

112. When to Remove Blockchain

Part XIII - Full Implementation Labs

113. Lab 1 - Build the Offline Installation Media

114. Lab 2 - Create the Three-Organization Fabric Network

115. Lab 3 - Deploy Shopping Chaincode

116. Lab 4 - Implement Product and Inventory Transactions

117. Lab 5 - Implement Checkout and Idempotency

118. Lab 6 - Build the Fabric Gateway Backend

119. Lab 7 - Build the Event Projection Worker

120. Lab 8 - Build the Local POS Web App

121. Lab 9 - Implement Returns and Refund Approvals

122. Lab 10 - Implement Receipt Hash Verification

123. Lab 11 - Secure the LAN and Identities

124. Lab 12 - Run Failure, Backup, and Recovery Drills

Appendix A - Recommended Repository Structure

Appendix B - Core TypeScript Chaincode Listing

Appendix C - Fabric Gateway Backend Listing

Appendix D - Docker Compose and Configuration Templates

Appendix E - REST API Specification

Appendix F - PostgreSQL Projection Schema

Appendix G - Test Matrix

Appendix H - Production Readiness Checklist

Appendix I - Offline Maintenance Runbook

Appendix J - Glossary and Official References

Part I - Architecture, Trust Model, and Blockchain Decisions

This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Chapter 1 - Why Use Blockchain in a Local Shopping System

Define the business reason for a permissioned ledger and avoid adding blockchain where a normal database is sufficient. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: Use blockchain for tamper-evident multi-party records, not for ordinary CRUD alone.

Implementation Objective

The concrete output of this chapter is: Blockchain decision memo. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use blockchain for tamper-evident multi-party records, not for ordinary CRUD alone.
- Distribute administrative control across Shop, Warehouse, and Audit roles if independent verification matters.
- Keep a conventional database for search, sessions, PII, reporting, and UI projections.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

1. Write the business objective in one sentence.
2. Identify the authoritative source for every data item.
3. Define which actors may create, read, approve, and correct the record.
4. Document the failure behavior before selecting technology.
5. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# Why Use Blockchain in a Local Shopping System
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The blockchain decision memo exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 2 - Reference Architecture for an Offline LAN Store

Establish the complete target architecture before selecting individual products or writing code. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: All runtime traffic stays inside the LAN.

Implementation Objective

The concrete output of this chapter is: Level-1 architecture diagram. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- All runtime traffic stays inside the LAN.
- Fabric peers and orderers form the trust layer; application services sit above Fabric Gateway.
- PostgreSQL and local object storage are off-chain systems, synchronized from committed ledger events.

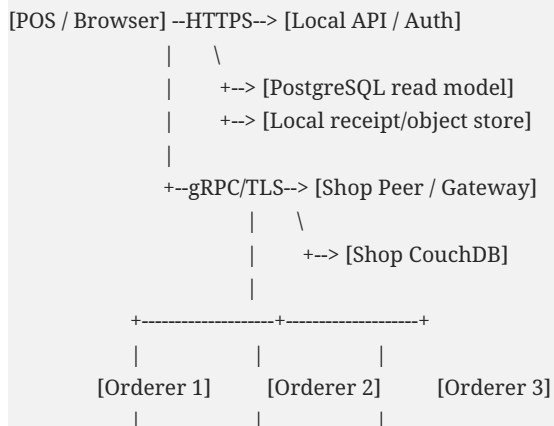
Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

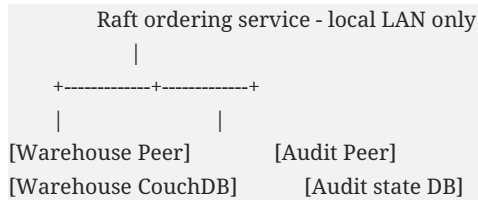
Implementation Procedure

6. Write the business objective in one sentence.
7. Identify the authoritative source for every data item.
8. Define which actors may create, read, approve, and correct the record.
9. Document the failure behavior before selecting technology.
10. Create an architecture decision record and review it with operations/security.

Implementation Example

LAN (no Internet required)





Verification Checklist

- The level-1 architecture diagram exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 3 - Trust Boundaries and Administrative Domains

Identify which people and systems are allowed to trust each other and which controls must remain independent. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: ShopOrg controls checkout and customer-facing operations.

Implementation Objective

The concrete output of this chapter is: Trust-boundary diagram. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- ShopOrg controls checkout and customer-facing operations.
- WarehouseOrg controls stock receipt and warehouse adjustments.
- AuditOrg maintains an independent ledger copy and read-focused access.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

11. Write the business objective in one sentence.
12. Identify the authoritative source for every data item.
13. Define which actors may create, read, approve, and correct the record.
14. Document the failure behavior before selecting technology.
15. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# Trust Boundaries and Administrative Domains
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The trust-boundary diagram exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 4 - What Goes On-Chain vs Off-Chain

Separate immutable business facts from mutable or private application data. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: Put order state transitions, stock movements, payment references, approvals, and hashes on-chain.

Implementation Objective

The concrete output of this chapter is: Data placement matrix. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Put order state transitions, stock movements, payment references, approvals, and hashes on-chain.
- Keep passwords, customer contact data, images, large receipts, and analytics tables off-chain.
- Store hashes/pointers on-chain when proof of an off-chain document is required.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

16. Write the business objective in one sentence.
17. Identify the authoritative source for every data item.
18. Define which actors may create, read, approve, and correct the record.
19. Document the failure behavior before selecting technology.
20. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# What Goes On-Chain vs Off-Chain
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The data placement matrix exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 5 - Permissioned Blockchain Concepts for Beginners

Explain peers, orderers, channels, identities, endorsements, world state, and immutable history in shopping-system language. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: Peers hold ledger copies and chaincode.

Implementation Objective

The concrete output of this chapter is: Concept map. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Peers hold ledger copies and chaincode.
- Orderers create a final ordered block stream.
- Membership Service Providers map X.509 identities to organizations and roles.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

21. Write the business objective in one sentence.
22. Identify the authoritative source for every data item.
23. Define which actors may create, read, approve, and correct the record.
24. Document the failure behavior before selecting technology.
25. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# Permissioned Blockchain Concepts for Beginners
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The concept map exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 6 - Fabric 2.5 LTS Baseline and Upgrade Strategy

Use a stable production baseline while keeping a documented path toward Fabric 3.x. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: Examples target Fabric 2.5.16 LTS and Fabric CA versions compatible with the official install tooling.

Implementation Objective

The concrete output of this chapter is: Version pinning policy. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Examples target Fabric 2.5.16 LTS and Fabric CA versions compatible with the official install tooling.
- Pin container digests and package versions for offline reproducibility.
- Evaluate Fabric 3.x features such as SmartBFT separately before migration.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

26. Write the business objective in one sentence.
27. Identify the authoritative source for every data item.
28. Define which actors may create, read, approve, and correct the record.
29. Document the failure behavior before selecting technology.
30. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# Fabric 2.5 LTS Baseline and Upgrade Strategy
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The version pinning policy exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 7 - Transaction Finality and Why It Matters for Retail

Show why checkout, returns, and stock changes need deterministic finality. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: A submitted transaction is endorsed, ordered, validated, and committed.

Implementation Objective

The concrete output of this chapter is: Transaction lifecycle diagram. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- A submitted transaction is endorsed, ordered, validated, and committed.
- Committed valid Fabric transactions are final; there is no proof-of-work fork competition.
- The application should wait for commit status before displaying irreversible success.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

31. Write the business objective in one sentence.
32. Identify the authoritative source for every data item.
33. Define which actors may create, read, approve, and correct the record.
34. Document the failure behavior before selecting technology.
35. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# Transaction Finality and Why It Matters for Retail
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The transaction lifecycle diagram exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 8 - Reference Non-Functional Requirements

Turn the system concept into measurable requirements. This chapter establishes an architectural decision that must be made before implementation. In a blockchain shopping system, the strongest designs clearly separate trust, immutable evidence, mutable application data, and operational convenience. The blockchain should be used where shared verification and tamper-evident history are valuable, while conventional local services remain responsible for user experience, indexing, reporting, and private data lifecycle.

Design rule: Define local availability target, acceptable checkout latency, backup RPO/RTO, and maximum outage window.

Implementation Objective

The concrete output of this chapter is: NFR checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Define local availability target, acceptable checkout latency, backup RPO/RTO, and maximum outage window.
- Specify that Internet loss must not affect normal sales operations.
- Define audit retention and certificate/key rotation requirements.

Document the decision as an ADR: context, options considered, selected option, consequences, security impact, operational impact, and reversal conditions. The ADR becomes part of the production evidence set and prevents later teams from treating blockchain-specific choices as accidental configuration.

Implementation Procedure

36. Write the business objective in one sentence.
37. Identify the authoritative source for every data item.
38. Define which actors may create, read, approve, and correct the record.
39. Document the failure behavior before selecting technology.
40. Create an architecture decision record and review it with operations/security.

Implementation Example

```
# Reference Non-Functional Requirements
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The nfr checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.

- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Blockchain is selected for marketing rather than a trust requirement.
- All nodes and keys remain controlled by one administrator, weakening the claimed multi-party assurance.
- Teams store every business field on-chain and later discover privacy/deletion problems.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part II - Shopping Domain and Ledger Data Model

The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Chapter 9 - Shopping Actors, Roles, and Responsibilities

Model customer, cashier, manager, warehouse operator, auditor, administrator, and service identities. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Customers do not need Fabric identities in the baseline design.

Implementation Objective

The concrete output of this chapter is: Role matrix. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Customers do not need Fabric identities in the baseline design.
- Staff actions that require attribution can use per-user or per-role Fabric client certificates.
- High-risk actions such as manual stock correction and refunds require additional authorization.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

41. Define stable entity identifiers and state transitions.
42. Classify every field as on-chain, off-chain, hash-only, or derived.
43. Define validation invariants and authorization ownership.
44. Create example records and edge cases.
45. Review privacy and retention implications before implementation.

Implementation Example

```
# Shopping Actors, Roles, and Responsibilities
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The role matrix exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 10 - Product Catalog Model

Design stable product identifiers, SKU rules, prices, tax fields, and lifecycle status. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Use immutable product IDs and unique SKUs.

Implementation Objective

The concrete output of this chapter is: Product schema. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use immutable product IDs and unique SKUs.
- Represent currency as integer minor units such as cents, never floating-point.
- Separate descriptive media and long text from core ledger facts.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

46. Define stable entity identifiers and state transitions.
47. Classify every field as on-chain, off-chain, hash-only, or derived.
48. Define validation invariants and authorization ownership.
49. Create example records and edge cases.
50. Review privacy and retention implications before implementation.

Implementation Example

```
# Product Catalog Model
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The product schema exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 11 - Inventory and Stock Movement Model

Model stock as an auditable sequence of receipts, sales, returns, reservations, releases, and corrections. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Keep onHand and reserved quantities as current state.

Implementation Objective

The concrete output of this chapter is: Inventory schema. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Keep onHand and reserved quantities as current state.
- Write explicit movement records keyed by transaction or movement ID.
- Never allow negative available stock unless an explicit backorder policy exists.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

51. Define stable entity identifiers and state transitions.
52. Classify every field as on-chain, off-chain, hash-only, or derived.
53. Define validation invariants and authorization ownership.
54. Create example records and edge cases.
55. Review privacy and retention implications before implementation.

Implementation Example

```
# Inventory and Stock Movement Model
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The inventory schema exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 12 - Cart and Reservation Model

Define how shopping carts reserve inventory without creating excessive blockchain traffic. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Keep transient carts off-chain by default.

Implementation Objective

The concrete output of this chapter is: Cart strategy. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Keep transient carts off-chain by default.
- Commit reservations only when scarce inventory or multi-terminal race prevention justifies it.
- Expire reservations deterministically using an application cleanup workflow plus ledger release transaction.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

56. Define stable entity identifiers and state transitions.
57. Classify every field as on-chain, off-chain, hash-only, or derived.
58. Define validation invariants and authorization ownership.
59. Create example records and edge cases.
60. Review privacy and retention implications before implementation.

Implementation Example

```
# Cart and Reservation Model
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The cart strategy exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 13 - Order Header and Order Line Model

Create a ledger representation that supports audit, lookup, refunds, and partial returns. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Use one order header plus line records or a bounded line array.

Implementation Objective

The concrete output of this chapter is: Order schemas. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use one order header plus line records or a bounded line array.
- Persist snapshot price/tax/discount values at purchase time.
- Never recompute historical totals from current product prices.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

61. Define stable entity identifiers and state transitions.
62. Classify every field as on-chain, off-chain, hash-only, or derived.
63. Define validation invariants and authorization ownership.
64. Create example records and edge cases.
65. Review privacy and retention implications before implementation.

Implementation Example

```
# Order Header and Order Line Model
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The order schemas exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 14 - Payment Recording Without Public Cryptocurrency

Record cash, local terminal, store credit, or external payment references without pretending the blockchain is a bank. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Store method, amount, status, reference hash, cashier/system identity, and timestamps.

Implementation Objective

The concrete output of this chapter is: Payment schema. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Store method, amount, status, reference hash, cashier/system identity, and timestamps.
- Do not put card PAN, CVV, or sensitive banking data on-chain.
- A payment transaction should be idempotent and tied to one order.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

66. Define stable entity identifiers and state transitions.
67. Classify every field as on-chain, off-chain, hash-only, or derived.
68. Define validation invariants and authorization ownership.
69. Create example records and edge cases.
70. Review privacy and retention implications before implementation.

Implementation Example

```
# Payment Recording Without Public Cryptocurrency
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The payment schema exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 15 - Refund and Return Model

Support full and partial returns with clear links back to original sale lines. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Return records reference original order and line IDs.

Implementation Objective

The concrete output of this chapter is: Return/refund schema. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Return records reference original order and line IDs.
- Inventory disposition distinguishes restockable, damaged, quarantine, and discarded items.
- Refund amount is calculated from historical line values, not current catalog price.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

71. Define stable entity identifiers and state transitions.
72. Classify every field as on-chain, off-chain, hash-only, or derived.
73. Define validation invariants and authorization ownership.
74. Create example records and edge cases.
75. Review privacy and retention implications before implementation.

Implementation Example

```
# Refund and Return Model
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The return/refund schema exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 16 - Discounts, Coupons, and Promotions

Make promotional pricing auditable while keeping evaluation practical. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Persist applied promotion ID and discount amount on the order.

Implementation Objective

The concrete output of this chapter is: Promotion decision table. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Persist applied promotion ID and discount amount on the order.
- Keep complex campaign targeting rules in the application unless network-wide consensus is required.
- Validate final totals in chaincode before accepting checkout.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

76. Define stable entity identifiers and state transitions.
77. Classify every field as on-chain, off-chain, hash-only, or derived.
78. Define validation invariants and authorization ownership.
79. Create example records and edge cases.
80. Review privacy and retention implications before implementation.

Implementation Example

```
# Discounts, Coupons, and Promotions
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The promotion decision table exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 17 - Receipts, Documents, and Hash Anchoring

Provide tamper evidence for printable receipts and local PDF invoices. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Generate receipt off-chain after commit.

Implementation Objective

The concrete output of this chapter is: Receipt hash workflow. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Generate receipt off-chain after commit.
- Hash the canonical receipt bytes and record the digest with the order or receipt record.
- Later verification recomputes the digest and compares it with ledger data.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

81. Define stable entity identifiers and state transitions.
82. Classify every field as on-chain, off-chain, hash-only, or derived.
83. Define validation invariants and authorization ownership.
84. Create example records and edge cases.
85. Review privacy and retention implications before implementation.

Implementation Example

```
# Receipts, Documents, and Hash Anchoring
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The receipt hash workflow exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 18 - Data Retention, Deletion, and Privacy Boundaries

Design around the fact that immutable ledger data cannot simply be deleted. The shopping domain must be modeled before code is written. Ledger records should represent business facts and state transitions that can be validated independently by every endorsing peer. The goal is not to copy a relational database into blockchain state; it is to encode the facts that need durable provenance and enforce invariants at transaction boundaries.

Design rule: Keep personal identifiers off-chain or pseudonymized.

Implementation Objective

The concrete output of this chapter is: Retention policy. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Keep personal identifiers off-chain or pseudonymized.
- Use revocable off-chain mappings when a customer relationship must be erased.
- Do not store unnecessary free-form notes in immutable state.

For every entity, define creation authority, update authority, terminal states, uniqueness rules, references to other entities, and whether an error is corrected by mutation or by a compensating record. Immutable systems work best when corrections are represented as new facts rather than silent rewrites.

Implementation Procedure

86. Define stable entity identifiers and state transitions.
87. Classify every field as on-chain, off-chain, hash-only, or derived.
88. Define validation invariants and authorization ownership.
89. Create example records and edge cases.
90. Review privacy and retention implications before implementation.

Implementation Example

```
# Data Retention, Deletion, and Privacy Boundaries
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The retention policy exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Money is stored as floating point.
- Historical orders are recalculated from current catalog values.
- State transitions are implicit strings with no validation.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part III - Offline LAN Infrastructure

An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Chapter 19 - Physical LAN Topology

Design switches, VLANs, servers, terminals, and administration paths for an Internet-independent deployment. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Use wired Ethernet for core peers/orderers where possible.

Implementation Objective

The concrete output of this chapter is: LAN topology. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use wired Ethernet for core peers/orderers where possible.
- Separate server, POS/client, management, and optional guest networks with VLANs/firewall rules.
- No component should require public DNS or cloud identity to start.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

91. Draw the physical and logical network.
92. List every port, hostname, certificate, image, package, and persistent volume.
93. Build installation artifacts on a controlled staging machine.
94. Install on an isolated test network.
95. Verify normal operation with the WAN disabled.

Implementation Example

```
# Physical LAN Topology
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The lan topology exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 20 - Small Single-Host Lab vs Multi-Server

Production

Choose a topology appropriate for learning, pilot, or resilient production. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Single host is acceptable for development but provides no meaningful fault isolation.

Implementation Objective

The concrete output of this chapter is: Deployment profiles. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Single host is acceptable for development but provides no meaningful fault isolation.
- Pilot can use three physical servers with distributed org services.
- Production should separate orderer quorum and peer storage according to failure domains.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

96. Draw the physical and logical network.
97. List every port, hostname, certificate, image, package, and persistent volume.
98. Build installation artifacts on a controlled staging machine.
99. Install on an isolated test network.
100. Verify normal operation with the WAN disabled.

Implementation Example

```
# Small Single-Host Lab vs Multi-Server Production
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The deployment profiles exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.

- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 21 - Local DNS and Hostname Strategy

Make certificate SANs and service discovery reliable without public DNS. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Run local DNS or static host mappings with stable names.

Implementation Objective

The concrete output of this chapter is: DNS plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Run local DNS or static host mappings with stable names.
- Issue TLS certificates for the exact internal DNS names clients use.
- Avoid localhost-only certificates outside development.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

101. Draw the physical and logical network.
102. List every port, hostname, certificate, image, package, and persistent volume.
103. Build installation artifacts on a controlled staging machine.
104. Install on an isolated test network.
105. Verify normal operation with the WAN disabled.

Implementation Example

```
# Local DNS and Hostname Strategy
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The dns plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 22 - Local Time Synchronization

Keep certificate validity, logs, and transaction diagnostics consistent without Internet NTP. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Run a local authoritative NTP/chrony source.

Implementation Objective

The concrete output of this chapter is: Time service plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Run a local authoritative NTP/chrony source.
- Synchronize all Fabric nodes, API servers, and database servers.
- Monitor clock drift because large differences complicate TLS and audit analysis.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

106. Draw the physical and logical network.
107. List every port, hostname, certificate, image, package, and persistent volume.
108. Build installation artifacts on a controlled staging machine.
109. Install on an isolated test network.
110. Verify normal operation with the WAN disabled.

Implementation Example

```
# Local Time Synchronization
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The time service plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 23 - Offline Software Supply Chain

Prepare every container image, binary, npm package, OS package, and checksum before moving into the air-gapped LAN. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Use an Internet-connected staging machine only to build the approved installation bundle.

Implementation Objective

The concrete output of this chapter is: Offline media manifest. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use an Internet-connected staging machine only to build the approved installation bundle.
- Record SHA-256 checksums and versions for every artifact.
- Import through controlled removable media or a one-way transfer process.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

111. Draw the physical and logical network.
112. List every port, hostname, certificate, image, package, and persistent volume.
113. Build installation artifacts on a controlled staging machine.
114. Install on an isolated test network.
115. Verify normal operation with the WAN disabled.

Implementation Example

```
# On the controlled Internet-connected staging workstation
mkdir -p offline-bundle/{images,binaries,npm,checksums}

docker pull hyperledger/fabric-peer:<PINNED_VERSION>
docker pull hyperledger/fabric-orderer:<PINNED_VERSION>
docker pull hyperledger/fabric-ca:<PINNED_CA_VERSION>
docker pull couchdb:<PINNED_VERSION>
docker save -o offline-bundle/images/fabric-images.tar \
  hyperledger/fabric-peer:<PINNED_VERSION> \
  hyperledger/fabric-orderer:<PINNED_VERSION> \
  hyperledger/fabric-ca:<PINNED_CA_VERSION> \
  couchdb:<PINNED_VERSION>

sha256sum offline-bundle/images/* > offline-bundle/checksums/SHA256SUMS
```

```
# On the isolated LAN host
docker load -i offline-bundle/images/fabric-images.tar
sha256sum -c offline-bundle/checksums/SHA256SUMS
```

Verification Checklist

- The offline media manifest exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 24 - Docker and Container Runtime in an Air-Gapped Environment

Install and operate containers without pulling images at runtime. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Use docker save/load or an internal registry.

Implementation Objective

The concrete output of this chapter is: Container operations runbook. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use docker save/load or an internal registry.
- Disable accidental public registry access.
- Use named volumes and bind mounts with documented ownership and backup rules.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

116. Draw the physical and logical network.
117. List every port, hostname, certificate, image, package, and persistent volume.
118. Build installation artifacts on a controlled staging machine.
119. Install on an isolated test network.
120. Verify normal operation with the WAN disabled.

Implementation Example

```
# Never rely on implicit pulls in production.
services:
  peer0-shop:
    image: hyperledger/fabric-peer:2.5.16
    pull_policy: never
    restart: unless-stopped
  volumes:
    - peer0_shop_data:/var/hyperledger/production
    - ./crypto/shop/peer0:/etc/hyperledger/fabric/msp:ro
```

Verification Checklist

- The container operations runbook exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.

- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 25 - Offline NPM and Application Dependency Management

Make Node.js builds reproducible without Internet access. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Commit package-lock.json.

Implementation Objective

The concrete output of this chapter is: Dependency cache procedure. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Commit package-lock.json.
- Pre-populate an offline npm cache or package tarball mirror.
- Use npm ci with locked dependencies and avoid floating version ranges in production builds.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

121. Draw the physical and logical network.
122. List every port, hostname, certificate, image, package, and persistent volume.
123. Build installation artifacts on a controlled staging machine.
124. Install on an isolated test network.
125. Verify normal operation with the WAN disabled.

Implementation Example

```
# Offline NPM and Application Dependency Management
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The dependency cache procedure exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.

- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 26 - Local Certificate and Secret Storage

Plan secure handling of CA keys, enrollment keys, TLS keys, API secrets, and database credentials. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Restrict private-key file permissions.

Implementation Objective

The concrete output of this chapter is: Secrets map. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Restrict private-key file permissions.
- Separate CA root/admin keys from day-to-day application identities.
- Back up secrets encrypted and test recovery.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

126. Draw the physical and logical network.
127. List every port, hostname, certificate, image, package, and persistent volume.
128. Build installation artifacts on a controlled staging machine.
129. Install on an isolated test network.
130. Verify normal operation with the WAN disabled.

Implementation Example

```
# Local Certificate and Secret Storage
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The secrets map exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 27 - Storage Layout and Filesystem Design

Separate immutable ledger blocks, world-state databases, application databases, logs, and backup staging. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Place high-write databases on durable storage with sufficient IOPS.

Implementation Objective

The concrete output of this chapter is: Disk layout. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Place high-write databases on durable storage with sufficient IOPS.
- Keep ledger and CouchDB/PostgreSQL backups coordinated.
- Monitor free space because peer/orderer failure due to full disks can halt operations.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

131. Draw the physical and logical network.
132. List every port, hostname, certificate, image, package, and persistent volume.
133. Build installation artifacts on a controlled staging machine.
134. Install on an isolated test network.
135. Verify normal operation with the WAN disabled.

Implementation Example

```
# Storage Layout and Filesystem Design
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The disk layout exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 28 - Local Monitoring and Administration Network

Operate health checks, dashboards, and log collection without cloud services. An offline deployment is an infrastructure discipline, not merely a missing Internet cable. Every runtime dependency, name-resolution path, certificate trust root, software artifact, clock source, backup destination, and administrative workflow must remain functional inside the local network. The design should assume that public DNS, package registries, cloud identity, telemetry endpoints, and vendor APIs are unavailable.

Design rule: Use local Prometheus/Grafana or equivalent if desired.

Implementation Objective

The concrete output of this chapter is: Monitoring architecture. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use local Prometheus/Grafana or equivalent if desired.
- Centralize logs on LAN with retention and access control.
- Provide an offline admin workstation with the required CLIs and certificates.

Create a machine-readable inventory containing hostname, static IP, VLAN, OS version, CPU/RAM/storage, Docker/container version, certificates, open ports, persistent volume paths, backup paths, and responsible owner. Keep this inventory in the offline documentation repository and update it after every infrastructure change.

Implementation Procedure

136. Draw the physical and logical network.
137. List every port, hostname, certificate, image, package, and persistent volume.
138. Build installation artifacts on a controlled staging machine.
139. Install on an isolated test network.
140. Verify normal operation with the WAN disabled.

Implementation Example

```
# Local Monitoring and Administration Network
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The monitoring architecture exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- A service silently depends on public DNS or a CDN.
- Runtime tries to pull a missing container/package from the Internet.
- Clock drift causes TLS or audit confusion.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part IV - Hyperledger Fabric Network Deployment

Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Chapter 29 - Fabric Installation Bundle for Offline Use

Build a reproducible package containing Fabric binaries, Fabric CA, Docker images, samples, and checksums. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Pin Fabric 2.5.x LTS versions.

Implementation Objective

The concrete output of this chapter is: Offline Fabric bundle. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Pin Fabric 2.5.x LTS versions.
- Export required images instead of relying on runtime pulls.
- Test the bundle on a clean isolated VM before production use.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

141. Generate/issue identities and trust roots.
142. Configure the component using version-controlled templates.
143. Start the service with persistent storage and TLS.
144. Join/configure the channel or lifecycle operation as required.
145. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Fabric Installation Bundle for Offline Use
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The offline fabric bundle exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 30 - Organization Design: ShopOrg, WarehouseOrg, AuditOrg, OrdererOrg

Map real administrative responsibilities to MSP organizations. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Keep org CA roots and admin identities separate.

Implementation Objective

The concrete output of this chapter is: Organization map. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Keep org CA roots and admin identities separate.
- AuditOrg can host an independent peer with read/audit applications.
- OrdererOrg operates the ordering service but should not control application endorsement alone.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

146. Generate/issue identities and trust roots.
147. Configure the component using version-controlled templates.
148. Start the service with persistent storage and TLS.
149. Join/configure the channel or lifecycle operation as required.
150. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Organization Design: ShopOrg, WarehouseOrg, AuditOrg, OrdererOrg
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The organization map exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 31 - Fabric CA Server Deployment

Create enrollment and TLS CAs for local identities. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Use TLS for CA endpoints even on a private LAN.

Implementation Objective

The concrete output of this chapter is: CA topology. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use TLS for CA endpoints even on a private LAN.
- Protect CA admin credentials and signing keys.
- Back up CA databases and configuration before identity lifecycle changes.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

151. Generate/issue identities and trust roots.
152. Configure the component using version-controlled templates.
153. Start the service with persistent storage and TLS.
154. Join/configure the channel or lifecycle operation as required.
155. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Fabric CA Server Deployment
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The ca topology exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.

- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 32 - Registering and Enrolling Identities

Issue X.509 identities for peers, orderers, admins, applications, and selected staff. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Registration defines name, secret, type, affiliation, and optional attributes.

Implementation Objective

The concrete output of this chapter is: Identity issuance workflow. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Registration defines name, secret, type, affiliation, and optional attributes.
- Enrollment creates a private key plus certificate.
- Use role attributes for chaincode authorization only after defining a controlled issuance process.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

156. Generate/issue identities and trust roots.
157. Configure the component using version-controlled templates.
158. Start the service with persistent storage and TLS.
159. Join/configure the channel or lifecycle operation as required.
160. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Registering and Enrolling Identities
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The identity issuance workflow exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 33 - MSP Directory Construction

Build local MSP structures that Fabric peers and orderers use to validate membership. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Separate signcerts, keystore, cacerts, tlscacerts, and config.yaml.

Implementation Objective

The concrete output of this chapter is: MSP layout. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Separate signcerts, keystore, cacerts, tlscacerts, and config.yaml.
- Enable NodeOUs where role-based certificate classification is required.
- Never copy private keys into shared read-only MSP directories unnecessarily.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

161. Generate/issue identities and trust roots.
162. Configure the component using version-controlled templates.
163. Start the service with persistent storage and TLS.
164. Join/configure the channel or lifecycle operation as required.
165. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# MSP Directory Construction
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The msp layout exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 34 - TLS Certificates for Internal Services

Protect gRPC communication between clients, peers, orderers, and CAs. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Issue SANs matching internal DNS names.

Implementation Objective

The concrete output of this chapter is: TLS certificate plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Issue SANs matching internal DNS names.
- Distribute trust roots explicitly.
- Rotate certificates with overlap and tested rollback procedures.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

166. Generate/issue identities and trust roots.
167. Configure the component using version-controlled templates.
168. Start the service with persistent storage and TLS.
169. Join/configure the channel or lifecycle operation as required.
170. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# TLS Certificates for Internal Services
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The tls certificate plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 35 - Deploying the Raft Ordering Service

Stand up a crash-fault-tolerant local ordering cluster. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Use an odd number of orderers, commonly three for a small resilient LAN.

Implementation Objective

The concrete output of this chapter is: Orderer deployment. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use an odd number of orderers, commonly three for a small resilient LAN.
- Persist orderer ledger data on durable volumes.
- Verify quorum behavior by stopping one node before go-live.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

171. Generate/issue identities and trust roots.
172. Configure the component using version-controlled templates.
173. Start the service with persistent storage and TLS.
174. Join/configure the channel or lifecycle operation as required.
175. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Deploying the Raft Ordering Service
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The orderer deployment exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.

- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 36 - Deploying Shop, Warehouse, and Audit Peers

Configure peers, gossip/delivery, state databases, TLS, and persistent storage. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Each peer has a local ledger copy for joined channels.

Implementation Objective

The concrete output of this chapter is: Peer deployment. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Each peer has a local ledger copy for joined channels.
- Use CouchDB when JSON rich queries/indexes are required; otherwise LevelDB can be simpler.
- Expose only required ports across VLANs.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

176. Generate/issue identities and trust roots.
177. Configure the component using version-controlled templates.
178. Start the service with persistent storage and TLS.
179. Join/configure the channel or lifecycle operation as required.
180. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Deploying Shop, Warehouse, and Audit Peers
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The peer deployment exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 37 - CouchDB State Database Design

Use CouchDB carefully for JSON-rich world state without treating it as the blockchain itself. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Create indexes packaged with chaincode for predictable queries.

Implementation Objective

The concrete output of this chapter is: CouchDB configuration. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Create indexes packaged with chaincode for predictable queries.
- Keep CouchDB colocated or on low-latency LAN segments.
- Do not back up CouchDB alone as if it were a complete Fabric ledger backup.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

181. Generate/issue identities and trust roots.
182. Configure the component using version-controlled templates.
183. Start the service with persistent storage and TLS.
184. Join/configure the channel or lifecycle operation as required.
185. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# CouchDB State Database Design
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The couchdb configuration exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 38 - Channel Configuration

Create a shopping channel with organizations, policies, capabilities, and orderer consenters. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: One shared shopping channel is a good baseline for the three organizations.

Implementation Objective

The concrete output of this chapter is: Channel configuration. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- One shared shopping channel is a good baseline for the three organizations.
- Use private data collections when subsets need confidentiality.
- Avoid unnecessary channels because every channel adds operational overhead.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

186. Generate/issue identities and trust roots.
187. Configure the component using version-controlled templates.
188. Start the service with persistent storage and TLS.
189. Join/configure the channel or lifecycle operation as required.
190. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Channel Configuration
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The channel configuration exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 39 - Creating and Joining the Shopping Channel

Create the channel and join peers/orderers using current system-channel-free tooling. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Create channel artifacts from approved configuration.

Implementation Objective

The concrete output of this chapter is: Channel bring-up runbook. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Create channel artifacts from approved configuration.
- Join each peer and verify channel membership.
- Confirm orderers actively service the channel before deploying chaincode.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

191. Generate/issue identities and trust roots.
192. Configure the component using version-controlled templates.
193. Start the service with persistent storage and TLS.
194. Join/configure the channel or lifecycle operation as required.
195. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Creating and Joining the Shopping Channel
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The channel bring-up runbook exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 40 - Anchor Peers, Discovery, and Gateway Connectivity

Ensure organizations can discover endpoints and client applications can transact reliably. Hyperledger Fabric provides the permissioned ledger layer. Network operators manage organizations, identities, peers, orderers, channels, policies, and lifecycle operations, while applications use Fabric Gateway to submit and evaluate smart-contract transactions. A production deployment should be scripted, version-pinned, recoverable, and auditable.

Design rule: Configure anchor peers where required by the chosen discovery/private-data topology.

Implementation Objective

The concrete output of this chapter is: Connectivity checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Configure anchor peers where required by the chosen discovery/private-data topology.
- Fabric Gateway in the peer simplifies endorsement and submission for clients.
- Test from the API server using the same DNS and TLS trust roots used in production.

Treat Fabric configuration as code. Store compose files, core/orderer/CA configuration, channel profiles, collection policies, and deployment scripts in version control. Operators should be able to rebuild a non-production environment entirely from the repository plus the offline artifact bundle.

Implementation Procedure

196. Generate/issue identities and trust roots.
197. Configure the component using version-controlled templates.
198. Start the service with persistent storage and TLS.
199. Join/configure the channel or lifecycle operation as required.
200. Run health/query/transaction tests and archive the resulting configuration.

Implementation Example

```
# Anchor Peers, Discovery, and Gateway Connectivity
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The connectivity checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Certificates do not match internal DNS names.
- Persistent volumes are omitted and a container rebuild loses state.
- Channel/chaincode configuration changes are performed manually without reproducible scripts.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part V - Smart Contracts and Chaincode

Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Chapter 41 - Chaincode Project Structure in TypeScript

Create a maintainable smart-contract codebase with contracts, models, validation, helpers, and tests. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Use fabric-contract-api for high-level contract methods.

Implementation Objective

The concrete output of this chapter is: Chaincode repository layout. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use fabric-contract-api for high-level contract methods.
- Keep pure domain calculations separate from Fabric-specific context code.
- Compile TypeScript before packaging and pin dependencies.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

201. Define input and output schemas.
202. Validate identity and authorization.
203. Read required ledger state and verify invariants.
204. Write state atomically and emit a compact event.
205. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Chaincode Project Structure in TypeScript
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The chaincode repository layout exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 42 - Deterministic Smart Contract Design

Ensure all endorsing peers compute the same result. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Do not use random numbers, external HTTP calls, or local wall-clock time inside chaincode.

Implementation Objective

The concrete output of this chapter is: Determinism checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Do not use random numbers, external HTTP calls, or local wall-clock time inside chaincode.
- Use the transaction timestamp provided by Fabric when a ledger timestamp is needed.
- Sort collections before serialization if ordering can vary.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

206. Define input and output schemas.
207. Validate identity and authorization.
208. Read required ledger state and verify invariants.
209. Write state atomically and emit a compact event.
210. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Deterministic Smart Contract Design
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The determinism checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 43 - Composite Key Strategy

Design keys for products, inventory, orders, lines, payments, movements, and idempotency records. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Prefix keys by entity type.

Implementation Objective

The concrete output of this chapter is: Key catalog. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Prefix keys by entity type.
- Use composite keys for secondary access patterns.
- Avoid unbounded scans in high-frequency transaction paths.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

211. Define input and output schemas.
212. Validate identity and authorization.
213. Read required ledger state and verify invariants.
214. Write state atomically and emit a compact event.
215. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
PRODUCT~{productId}
SKU~{sku} -> productId
INV~{storeId}~{productId}
ORDER~{orderId}
ORDERLINE~{orderId}~{lineNo}
PAYMENT~{paymentId}
RETURN~{returnId}
MOVE~{storeId}~{productId}~{movementId}
REQUEST~{clientRequestId}
RECEIPT~{orderId}
```

Verification Checklist

- The key catalog exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.

- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 44 - Product Contract Implementation

Create, update, deactivate, and read auditable product master facts. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Reject duplicate product IDs/SKUs.

Implementation Objective

The concrete output of this chapter is: Product contract. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Reject duplicate product IDs/SKUs.
- Validate price and tax values as integers/ranges.
- Record who performed administrative changes through transaction identity and event metadata.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

216. Define input and output schemas.
217. Validate identity and authorization.
218. Read required ledger state and verify invariants.
219. Write state atomically and emit a compact event.
220. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
import { Context, Contract, Info, Transaction } from 'fabric-contract-api';

@Info({ title: 'ShoppingContract' })
export class ShoppingContract extends Contract {
  @Transaction()
  public async createProduct(
    ctx: Context,
    productId: string,
    sku: string,
    name: string,
    priceCentsText: string,
    taxBpsText: string,
  ): Promise<void> {
    this.requireRole(ctx, ['manager']);
    const key = `PRODUCT~${productId}`;
    if ((await ctx.stub.getState(key)).length) throw new Error('PRODUCT_EXISTS');
```

```

const priceCents = Number(priceCentsText);
const taxBps = Number(taxBpsText);
if (!Number.isSafeInteger(priceCents) || priceCents < 0) throw new Error('BAD_PRICE');
if (!Number.isSafeInteger(taxBps) || taxBps < 0 || taxBps > 10000) throw new Error('BAD_TAX');

const product = { productId, sku, name, priceCents, taxBps, active: true };
await ctx.stub.putState(key, Buffer.from(JSON.stringify(product)));
ctx.stub.setEvent('ProductChanged', Buffer.from(JSON.stringify({ productId })));
}
}

```

Verification Checklist

- The product contract exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 45 - Inventory Contract Implementation

Receive, reserve, release, sell, return, and adjust stock with invariant checks. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: All quantity updates must pass non-negative/invariant rules.

Implementation Objective

The concrete output of this chapter is: Inventory contract. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- All quantity updates must pass non-negative/invariant rules.
- Create a StockMovement record for every change.
- Use optimistic concurrency/MVCC behavior instead of inventing application-side locks.

Return stable business error codes such as PRODUCT_EXISTS, OUT_OF_STOCK, INVALID_STATE, FORBIDDEN, or DUPLICATE_REQUEST. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

221. Define input and output schemas.
222. Validate identity and authorization.
223. Read required ledger state and verify invariants.
224. Write state atomically and emit a compact event.
225. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Inventory Contract Implementation
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The inventory contract exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 46 - Order Contract Implementation

Create orders, add immutable purchase snapshots, and enforce allowed status transitions. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Validate line totals and order totals independently in chaincode.

Implementation Objective

The concrete output of this chapter is: Order contract. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Validate line totals and order totals independently in chaincode.
- Disallow editing completed orders; model changes as new events such as return/refund.
- Use explicit status-transition rules rather than arbitrary string assignment.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

226. Define input and output schemas.
227. Validate identity and authorization.
228. Read required ledger state and verify invariants.
229. Write state atomically and emit a compact event.
230. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Order Contract Implementation
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The order contract exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 47 - Payment Record Contract

Link payment facts to orders without storing sensitive payment credentials. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Use unique payment IDs and idempotency keys.

Implementation Objective

The concrete output of this chapter is: Payment contract. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use unique payment IDs and idempotency keys.
- Validate total paid against order balance rules.
- Represent void/refund as new records and status transitions, not destructive edits.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

231. Define input and output schemas.
232. Validate identity and authorization.
233. Read required ledger state and verify invariants.
234. Write state atomically and emit a compact event.
235. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Payment Record Contract
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The payment contract exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 48 - Return and Refund Contract

Handle partial returns, restocking, damaged goods, and refund authorization. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Reference original order line and quantity.

Implementation Objective

The concrete output of this chapter is: Return contract. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Reference original order line and quantity.
- Prevent returned quantity from exceeding purchased quantity minus prior returns.
- Separate inventory disposition from financial refund decision.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

236. Define input and output schemas.
237. Validate identity and authorization.
238. Read required ledger state and verify invariants.
239. Write state atomically and emit a compact event.
240. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Return and Refund Contract
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The return contract exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 49 - Role and Attribute Authorization in Chaincode

Enforce organization and role permissions close to the ledger update. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Check MSP ID for organizational boundary.

Implementation Objective

The concrete output of this chapter is: Authorization helper. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Check MSP ID for organizational boundary.
- Use certificate attributes for roles only when CA issuance is controlled.
- Keep API authorization too; chaincode is the final enforcement layer for ledger writes.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

241. Define input and output schemas.
242. Validate identity and authorization.
243. Read required ledger state and verify invariants.
244. Write state atomically and emit a compact event.
245. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
private requireRole(ctx: Context, roles: string[]): void {
  const mspId = ctx.clientIdentity.getMSPID();
  const role = ctx.clientIdentity.getAttributeValue('role');
  if (!role || !roles.includes(role)) {
    throw new Error(`FORBIDDEN:${mspId}:${role ?? 'none'}`);
  }
}
```

Verification Checklist

- The authorization helper exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 50 - Business Rule Validation

Encode rules that must hold regardless of which application client submits a transaction. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Validate identifiers, money, quantity, state transitions, and references.

Implementation Objective

The concrete output of this chapter is: Validation library. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Validate identifiers, money, quantity, state transitions, and references.
- Reject malformed JSON and oversized free-form inputs.
- Return stable error codes for API mapping.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

246. Define input and output schemas.
247. Validate identity and authorization.
248. Read required ledger state and verify invariants.
249. Write state atomically and emit a compact event.
250. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Business Rule Validation
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The validation library exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 51 - Idempotency at Ledger Level

Prevent duplicate checkout, payment, and refund transactions caused by retries. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Require a `clientRequestId` for side-effecting operations.

Implementation Objective

The concrete output of this chapter is: Idempotency pattern. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Require a `clientRequestId` for side-effecting operations.
- Create a request marker key atomically with the business update.
- If a marker already exists, return the prior result or a duplicate-safe response.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

251. Define input and output schemas.
252. Validate identity and authorization.
253. Read required ledger state and verify invariants.
254. Write state atomically and emit a compact event.
255. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
const requestKey = `REQUEST~${clientRequestId}`;
const prior = await ctx.stub.getState(requestKey);
if (prior.length) {
  return prior.toString('utf8'); // duplicate-safe response
}

// Perform business writes here...
const result = JSON.stringify({ orderId, status: 'COMMITTED' });
await ctx.stub.putState(requestKey, Buffer.from(result));
return result;
```

Verification Checklist

- The idempotency pattern exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.

- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 52 - MVCC and Concurrent Checkout

Use Fabric version checks to safely resolve concurrent stock updates. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Two checkouts reading the same stock may produce an MVCC conflict for one transaction.

Implementation Objective

The concrete output of this chapter is: Concurrency test plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Two checkouts reading the same stock may produce an MVCC conflict for one transaction.
- Retry only after re-reading the latest state.
- Do not blindly resubmit stale proposals.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

256. Define input and output schemas.
257. Validate identity and authorization.
258. Read required ledger state and verify invariants.
259. Write state atomically and emit a compact event.
260. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# MVCC and Concurrent Checkout
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The concurrency test plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 53 - Chaincode Events

Emit business events for projections, receipts, notifications, and audit workflows. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Events are emitted from successful chaincode execution and observed after commit by applications.

Implementation Objective

The concrete output of this chapter is: Event catalog. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Events are emitted from successful chaincode execution and observed after commit by applications.
- Keep payloads compact and include stable identifiers.
- Consumers must still be idempotent because event delivery/reconnect can replay data.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

261. Define input and output schemas.
262. Validate identity and authorization.
263. Read required ledger state and verify invariants.
264. Write state atomically and emit a compact event.
265. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Chaincode Events
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The event catalog exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 54 - Rich Queries and CouchDB Indexes

Support product, order, and audit queries without harming transaction correctness. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Use rich queries mainly for read/report flows.

Implementation Objective

The concrete output of this chapter is: Index definitions. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use rich queries mainly for read/report flows.
- Package JSON indexes under META-INF/statedb/couchdb/indexes.
- Do not rely on a non-repeatable rich query result as the sole basis for complex concurrent updates.

Return stable business error codes such as PRODUCT_EXISTS, OUT_OF_STOCK, INVALID_STATE, FORBIDDEN, or DUPLICATE_REQUEST. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

266. Define input and output schemas.
267. Validate identity and authorization.
268. Read required ledger state and verify invariants.
269. Write state atomically and emit a compact event.
270. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Rich Queries and CouchDB Indexes
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The index definitions exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 55 - Private Data Collections

Store selected confidential organization-to-organization facts without putting plaintext on the channel ledger for every member. Chaincode is the final business-rule enforcement layer for ledger updates. It must be deterministic, validate all untrusted input, enforce authorization, and preserve invariants even if a client application is buggy or malicious. Because endorsing peers execute the same transaction proposal independently, nondeterministic logic or external calls must be excluded.

Design rule: Use PDCs only for data that must be shared with a subset of channel organizations.

Implementation Objective

The concrete output of this chapter is: Private data design. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use PDCs only for data that must be shared with a subset of channel organizations.
- All required members must approve matching collection definitions with chaincode lifecycle.
- Remember that hashes of private data are still committed for proof.

Return stable business error codes such as `PRODUCT_EXISTS`, `OUT_OF_STOCK`, `INVALID_STATE`, `FORBIDDEN`, or `DUPLICATE_REQUEST`. The API can translate these into HTTP status codes and user-friendly messages without parsing arbitrary error text. Keep all state writes for a logical business transaction inside one chaincode transaction whenever atomicity is required.

Implementation Procedure

271. Define input and output schemas.
272. Validate identity and authorization.
273. Read required ledger state and verify invariants.
274. Write state atomically and emit a compact event.
275. Test success, invalid input, unauthorized caller, duplicate request, and concurrency conflict.

Implementation Example

```
# Private Data Collections
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The private data design exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- External API calls or nondeterministic values are used inside chaincode.
- Authorization exists only in the UI/API but not for critical ledger writes.
- Retries create duplicate orders/payments because no idempotency key is enforced.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part VI - Backend API and Fabric Gateway Integration

The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Chapter 56 - Backend Service Architecture

Create the LAN API service that authenticates users, validates requests, calls Fabric Gateway, and exposes REST endpoints. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Browser/POS clients never receive peer private keys.

Implementation Objective

The concrete output of this chapter is: Backend component diagram. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Browser/POS clients never receive peer private keys.
- The backend maps authenticated application users to appropriate Fabric signing identities.
- Separate query endpoints from side-effecting command endpoints.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

276. Define the REST command/query contract.
277. Authenticate and authorize the user.
278. Validate payload and create a correlation/idempotency key.
279. Call Fabric Gateway and wait for commit for writes.
280. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Backend Service Architecture
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The backend component diagram exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 57 - Node.js/TypeScript Project Bootstrap

Create an offline-installable backend using Express, TypeScript, PostgreSQL, and Fabric Gateway. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Pin dependencies with package-lock.json.

Implementation Objective

The concrete output of this chapter is: package.json and folder layout. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Pin dependencies with package-lock.json.
- Compile into a deployable dist directory.
- Load configuration from local environment files or a secret store, never source control.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

281. Define the REST command/query contract.
282. Authenticate and authorize the user.
283. Validate payload and create a correlation/idempotency key.
284. Call Fabric Gateway and wait for commit for writes.
285. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Node.js/TypeScript Project Bootstrap
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The package.json and folder layout exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 58 - Fabric Gateway Connection Helper

Open gRPC/TLS connections, create identities/signers, and obtain a contract handle. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Use @hyperledger/fabric-gateway.

Implementation Objective

The concrete output of this chapter is: gateway.ts. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use @hyperledger/fabric-gateway.
- Set realistic evaluate/endorse/submit/commit deadlines.
- Reuse gateway/client connections instead of reconnecting for every HTTP request.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

286. Define the REST command/query contract.
287. Authenticate and authorize the user.
288. Validate payload and create a correlation/idempotency key.
289. Call Fabric Gateway and wait for commit for writes.
290. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
import * as grpc from '@grpc/grpc-js';
import { connect, hash, signers } from '@hyperledger/fabric-gateway';
import { readFile } from 'node:fs/promises';
import { createPrivateKey } from 'node:crypto';

const tlsRootCert = await readFile(process.env.PEER_TLS_CA!);
const client = new grpc.Client(process.env.PEER_ENDPOINT!,
  grpc.credentials.createSsl(tlsRootCert));

const credentials = await readFile(process.env.CLIENT_CERT!);
const privateKeyPem = await readFile(process.env.CLIENT_KEY!);
const privateKey = createPrivateKey(privateKeyPem);

const gateway = connect({
  client,
  identity: { mspId: process.env.MSP_ID!, credentials },
  signer: signers.newPrivateKeySigner(privateKey),
  hash: hash.sha256,
```

```

evaluateOptions: () => ({ deadline: Date.now() + 5_000 }),
endorseOptions: () => ({ deadline: Date.now() + 10_000 }),
submitOptions: () => ({ deadline: Date.now() + 10_000 }),
commitStatusOptions: () => ({ deadline: Date.now() + 30_000 }),
});

const network = gateway.getNetwork('shopping');
export const contract = network.getContract('shoppingcc');

```

Verification Checklist

- The gateway.ts exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 59 - Identity Mapping for Staff and Services

Choose between per-user certificates, per-role identities, and service identities. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Per-user identities improve cryptographic attribution but increase certificate operations.

Implementation Objective

The concrete output of this chapter is: Identity mapping policy. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Per-user identities improve cryptographic attribution but increase certificate operations.
- Service identities simplify retail frontends; include authenticated operator references off-chain and in permitted ledger fields.
- Do not let a browser choose arbitrary MSP identity files.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

291. Define the REST command/query contract.
292. Authenticate and authorize the user.
293. Validate payload and create a correlation/idempotency key.
294. Call Fabric Gateway and wait for commit for writes.
295. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Identity Mapping for Staff and Services
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The identity mapping policy exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 60 - Authentication and Session Management

Authenticate local users without cloud identity dependencies. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Use salted password hashes or local LDAP/SSO.

Implementation Objective

The concrete output of this chapter is: Auth subsystem. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use salted password hashes or local LDAP/SSO.
- Use short-lived signed sessions/JWTs plus refresh or re-login policy appropriate for terminals.
- Separate customer accounts from privileged staff accounts.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

296. Define the REST command/query contract.
297. Authenticate and authorize the user.
298. Validate payload and create a correlation/idempotency key.
299. Call Fabric Gateway and wait for commit for writes.
300. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Authentication and Session Management
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The auth subsystem exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 61 - Product REST API

Expose product creation/admin and customer-safe product reads. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Admin writes submit ledger transactions.

Implementation Objective

The concrete output of this chapter is: REST endpoints. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Admin writes submit ledger transactions.
- Read APIs can serve from PostgreSQL projection for speed while providing ledger verification paths.
- Validate request schemas before invoking Fabric.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

301. Define the REST command/query contract.
302. Authenticate and authorize the user.
303. Validate payload and create a correlation/idempotency key.
304. Call Fabric Gateway and wait for commit for writes.
305. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Product REST API
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The rest endpoints exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 62 - Inventory REST API

Expose stock receipt, availability, adjustment, and movement history. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Critical adjustments require manager authorization.

Implementation Objective

The concrete output of this chapter is: Inventory endpoints. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Critical adjustments require manager authorization.
- Inventory availability displayed to users should clearly distinguish onHand, reserved, and available.
- Conflict errors should trigger a controlled re-read/retry flow.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

306. Define the REST command/query contract.
307. Authenticate and authorize the user.
308. Validate payload and create a correlation/idempotency key.
309. Call Fabric Gateway and wait for commit for writes.
310. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Inventory REST API
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The inventory endpoints exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 63 - Order and Checkout REST API

Implement quote validation, checkout submission, commit wait, and receipt generation. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Server recalculates totals; never trust browser totals.

Implementation Objective

The concrete output of this chapter is: Checkout endpoint. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Server recalculates totals; never trust browser totals.
- Use an idempotency key from the client.
- Return success only after the transaction has committed validly.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

311. Define the REST command/query contract.
312. Authenticate and authorize the user.
313. Validate payload and create a correlation/idempotency key.
314. Call Fabric Gateway and wait for commit for writes.
315. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
app.post('/api/orders/checkout', requireUser, async (req, res) => {
  const requestId = req.header('Idempotency-Key');
  if (!requestId) return res.status(400).json({ code: 'IDEMPOTENCY_REQUIRED' });

  const input = checkoutSchema.parse(req.body);
  const resultBytes = await contract.submitTransaction(
    'Checkout', requestId, JSON.stringify(input)
  );
  const result = JSON.parse(Buffer.from(resultBytes).toString('utf8'));
  res.status(201).json(result);
});
```

Verification Checklist

- The checkout endpoint exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.

- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 64 - Payment and Refund REST API

Record local payment outcomes and controlled refund requests. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Never store raw card secrets.

Implementation Objective

The concrete output of this chapter is: Payment endpoints. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Never store raw card secrets.
- Separate payment capture/terminal interaction from ledger recording when a local terminal exists.
- Require explicit authorization for refunds over configured thresholds.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

316. Define the REST command/query contract.
317. Authenticate and authorize the user.
318. Validate payload and create a correlation/idempotency key.
319. Call Fabric Gateway and wait for commit for writes.
320. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# Payment and Refund REST API
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The payment endpoints exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 65 - Chaincode Event Listener and Projection Worker

Build a resilient local process that consumes committed events and updates PostgreSQL/read models. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Checkpoint the last processed block/event position.

Implementation Objective

The concrete output of this chapter is: Projection service. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Checkpoint the last processed block/event position.
- Make each projection operation idempotent.
- A projection failure must not roll back the already-committed blockchain transaction.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

321. Define the REST command/query contract.
322. Authenticate and authorize the user.
323. Validate payload and create a correlation/idempotency key.
324. Call Fabric Gateway and wait for commit for writes.
325. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
const events = await network.getChaincodeEvents('shoppingcc', {
  startBlock: checkpoint.nextBlock,
});

for await (const event of events) {
  await db.tx(async t => {
    if (await alreadyProcessed(t, event.transactionId, event.eventName)) return;
    await applyProjection(t, event);
    await saveCheckpoint(t, event.blockNumber + 1n);
  });
}
```

Verification Checklist

- The projection service exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.

- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 66 - PostgreSQL Read Model

Create fast searchable tables for UI queries and reporting while treating ledger transactions as authoritative for audited facts. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Use foreign keys and indexes for operational reads.

Implementation Objective

The concrete output of this chapter is: SQL schema. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use foreign keys and indexes for operational reads.
- Store ledger txId/block references with projected rows.
- Provide rebuild tooling that can recreate projections from ledger events or exports.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

326. Define the REST command/query contract.
327. Authenticate and authorize the user.
328. Validate payload and create a correlation/idempotency key.
329. Call Fabric Gateway and wait for commit for writes.
330. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
CREATE TABLE orders (
  order_id    text PRIMARY KEY,
  status      text NOT NULL,
  total_cents bigint NOT NULL,
  payment_status text NOT NULL,
  ledger_tx_id text NOT NULL,
  ledger_block_no bigint,
  updated_at  timestampz NOT NULL DEFAULT now()
);

CREATE INDEX idx_orders_status ON orders(status);
```

Verification Checklist

- The sql schema exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.

- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 67 - API Error Model, Correlation IDs, and Logging

Make failures diagnosable across HTTP, gateway, peer, orderer, and database layers. The backend is the integration boundary between ordinary business applications and the Fabric network. It authenticates users, validates HTTP requests, maps them to appropriate Fabric identities, waits for commit results, maintains idempotency, consumes ledger events, and exposes stable APIs to local clients.

Design rule: Assign one correlation ID per business request.

Implementation Objective

The concrete output of this chapter is: Error schema. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Assign one correlation ID per business request.
- Log txId when available.
- Map deterministic chaincode errors to stable application error codes and keep internal stack traces out of client responses.

The backend should use command/query separation. Queries can often read a fast projection, while commands submit a ledger transaction and wait for commit. If an HTTP connection is lost after submission, the idempotency key and transaction correlation data allow the client to determine whether the operation committed before retrying.

Implementation Procedure

331. Define the REST command/query contract.
332. Authenticate and authorize the user.
333. Validate payload and create a correlation/idempotency key.
334. Call Fabric Gateway and wait for commit for writes.
335. Persist/log only off-chain data and return a stable client response.

Implementation Example

```
# API Error Model, Correlation IDs, and Logging
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The error schema exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- API returns success before ledger commit.
- The browser is given Fabric private keys.
- Projection database is accidentally treated as authoritative after falling behind.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part VII - Local Web/POS User Interface

The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Chapter 68 - Frontend Architecture for LAN-Only Operation

Build a web UI that requires only the local API and locally hosted static assets. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: Bundle JavaScript, CSS, fonts, and icons locally.

Implementation Objective

The concrete output of this chapter is: Frontend architecture. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Bundle JavaScript, CSS, fonts, and icons locally.
- Avoid CDN dependencies.
- Use relative/local API URLs and health indicators for local services.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

336. Define the user task and allowed states.
337. Load data through the local API only.
338. Disable or gate unsafe actions based on authorization and transaction state.
339. Handle pending, conflict, validation, and service-unavailable states explicitly.
340. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Build once on the offline build server.
npm ci --offline
npm run build

# Serve dist/ from the local API reverse proxy.
# No Google Fonts, CDN JS, analytics, or public API calls.
```

Verification Checklist

- The frontend architecture exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 69 - Login and Role-Aware Navigation

Present customer/cashier/manager/warehouse/auditor screens based on authenticated authorization. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: UI hiding is convenience, not security.

Implementation Objective

The concrete output of this chapter is: Navigation matrix. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- UI hiding is convenience, not security.
- Backend and chaincode enforce the real permissions.
- Show current signed-in role and terminal/store identity.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

341. Define the user task and allowed states.
342. Load data through the local API only.
343. Disable or gate unsafe actions based on authorization and transaction state.
344. Handle pending, conflict, validation, and service-unavailable states explicitly.
345. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Login and Role-Aware Navigation
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The navigation matrix exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 70 - Catalog and Product Search

Serve fast product browsing from local projections while preserving ledger auditability. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: Use indexed PostgreSQL queries for search.

Implementation Objective

The concrete output of this chapter is: Catalog screens. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use indexed PostgreSQL queries for search.
- Display product availability with the timestamp/source of the projection.
- Offer an admin verification action for critical records if needed.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

346. Define the user task and allowed states.
347. Load data through the local API only.
348. Disable or gate unsafe actions based on authorization and transaction state.
349. Handle pending, conflict, validation, and service-unavailable states explicitly.
350. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Catalog and Product Search
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The catalog screens exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 71 - Cart and Checkout UX

Design retry-safe checkout on unreliable terminals or LAN links. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: Disable double-submit in the browser and still enforce server/ledger idempotency.

Implementation Objective

The concrete output of this chapter is: Checkout state machine. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Disable double-submit in the browser and still enforce server/ledger idempotency.
- Show pending while commit status is unknown.
- Never display final receipt number before commit confirmation.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

351. Define the user task and allowed states.
352. Load data through the local API only.
353. Disable or gate unsafe actions based on authorization and transaction state.
354. Handle pending, conflict, validation, and service-unavailable states explicitly.
355. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Cart and Checkout UX
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The checkout state machine exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 72 - Cashier/POS Workflow

Optimize scan, quantity, payment, commit, and receipt flow for local terminals. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: Support barcode input as keyboard/scanner events.

Implementation Objective

The concrete output of this chapter is: POS workflow. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Support barcode input as keyboard/scanner events.
- Keep the critical path short.
- Provide a recovery screen for transactions whose HTTP response was lost after ledger commit.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

356. Define the user task and allowed states.
357. Load data through the local API only.
358. Disable or gate unsafe actions based on authorization and transaction state.
359. Handle pending, conflict, validation, and service-unavailable states explicitly.
360. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Cashier/POS Workflow
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The pos workflow exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 73 - Warehouse and Inventory Screens

Give warehouse staff controlled stock receiving and adjustment tools. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: Require reason codes for adjustments.

Implementation Objective

The concrete output of this chapter is: Warehouse UI. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Require reason codes for adjustments.
- Show movement history and ledger transaction IDs.
- Use confirmation for high-impact operations.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

361. Define the user task and allowed states.
362. Load data through the local API only.
363. Disable or gate unsafe actions based on authorization and transaction state.
364. Handle pending, conflict, validation, and service-unavailable states explicitly.
365. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Warehouse and Inventory Screens
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The warehouse ui exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 74 - Audit and Verification Dashboard

Allow auditors/managers to verify order history, receipt hashes, stock movements, and identities. The user interface should feel like a normal shopping/POS system even though critical business facts are committed through blockchain. Blockchain-specific complexity belongs behind the API. The UI focuses on clear state, safe retry behavior, role-appropriate actions, and recovery when a request is pending or the network is partially unavailable.

Design rule: Show transaction IDs, timestamps, signer MSP/role where appropriate, and status history.

Implementation Objective

The concrete output of this chapter is: Audit UI. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Show transaction IDs, timestamps, signer MSP/role where appropriate, and status history.
- Provide filters without directly exposing admin credentials.
- Export signed audit reports only from trusted server-side processes.

Every write action should have an explicit UI lifecycle: ready -> submitting -> pending commit -> committed or rejected. Operators should be able to recover from a browser refresh or terminal reconnect without guessing whether a sale succeeded. The UI should show a business reference and transaction reference for support staff.

Implementation Procedure

366. Define the user task and allowed states.
367. Load data through the local API only.
368. Disable or gate unsafe actions based on authorization and transaction state.
369. Handle pending, conflict, validation, and service-unavailable states explicitly.
370. Test with real operators and slow/failing network scenarios.

Implementation Example

```
# Audit and Verification Dashboard
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The audit ui exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Double-click creates multiple requests.
- A pending commit is displayed as failed and the operator retries, causing confusion.
- The UI loads CDN fonts/scripts and breaks when offline.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part VIII - End-to-End Business Flows

An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Chapter 75 - End-to-End Product Creation Flow

Trace product administration from UI to committed ledger and projection. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Admin authenticates, backend validates, chaincode commits, event updates projection, UI reads projected state.

Implementation Objective

The concrete output of this chapter is: Sequence diagram. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Admin authenticates, backend validates, chaincode commits, event updates projection, UI reads projected state.
- If projection is delayed, the committed ledger remains authoritative.
- Every step shares a correlation/transaction reference.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

371. Start from the user/business trigger.
372. Trace identity and validation at each boundary.
373. Mark the ledger transaction boundary and commit confirmation.
374. Define event/projection side effects after commit.
375. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# End-to-End Product Creation Flow
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The sequence diagram exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.
- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 76 - Stock Receiving Flow

Implement warehouse receipt with cross-checks and optional dual endorsement. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Receive shipment reference off-chain or hashed.

Implementation Objective

The concrete output of this chapter is: Sequence diagram. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Receive shipment reference off-chain or hashed.
- Warehouse submits stock receipt; chaincode updates inventory and movement.
- Optional state-based endorsement can require ShopOrg plus WarehouseOrg for selected inventory keys.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

376. Start from the user/business trigger.
377. Trace identity and validation at each boundary.
378. Mark the ledger transaction boundary and commit confirmation.
379. Define event/projection side effects after commit.
380. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# Stock Receiving Flow
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The sequence diagram exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.
- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 77 - Customer Checkout Flow

Implement cart validation, inventory decrement, order creation, payment record, and receipt creation. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Re-read prices/availability on server immediately before submit.

Implementation Objective

The concrete output of this chapter is: Checkout sequence. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Re-read prices/availability on server immediately before submit.
- Write order and stock movement in one atomic chaincode transaction where feasible.
- Generate receipt only after valid commit.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

381. Start from the user/business trigger.
382. Trace identity and validation at each boundary.
383. Mark the ledger transaction boundary and commit confirmation.
384. Define event/projection side effects after commit.
385. Exercise rollback/retry/recovery for every failure point.

Implementation Example

1. Browser sends cart + Idempotency-Key to local API.
2. API authenticates user and re-reads authoritative product/price data.
3. API submits Checkout to Fabric Gateway.
4. Endorsing peers execute chaincode; inventory/order invariants are checked.
5. Raft orderers order the transaction.
6. Peers validate and commit.
7. Gateway returns commit status to API.
8. API returns success/txId; receipt is generated locally.
9. Chaincode event updates PostgreSQL projection.

Verification Checklist

- The checkout sequence exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.
- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 78 - Concurrent Checkout Conflict Recovery

Handle two terminals trying to sell the last item. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: One transaction may commit while another is invalidated by version conflict.

Implementation Objective

The concrete output of this chapter is: Conflict workflow. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- One transaction may commit while another is invalidated by version conflict.
- Backend re-reads stock and reports an out-of-stock or retryable result.
- Never manufacture stock to make the second checkout succeed.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

386. Start from the user/business trigger.
387. Trace identity and validation at each boundary.
388. Mark the ledger transaction boundary and commit confirmation.
389. Define event/projection side effects after commit.
390. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# Concurrent Checkout Conflict Recovery
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The conflict workflow exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.

- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 79 - Return and Refund Flow

Implement item validation, approval, inventory disposition, refund record, and customer receipt. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Look up original ledger purchase.

Implementation Objective

The concrete output of this chapter is: Return sequence. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Look up original ledger purchase.
- Require reason and authorization according to policy.
- Write return/refund facts as new immutable records.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

391. Start from the user/business trigger.
392. Trace identity and validation at each boundary.
393. Mark the ledger transaction boundary and commit confirmation.
394. Define event/projection side effects after commit.
395. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# Return and Refund Flow
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The return sequence exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.
- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 80 - Order Cancellation and Reservation Release

Cancel allowed orders without rewriting history. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Only orders in allowed pre-completion states can cancel.

Implementation Objective

The concrete output of this chapter is: Cancellation state machine. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Only orders in allowed pre-completion states can cancel.
- Release any ledger reservations atomically.
- Keep cancellation reason and actor identity.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

396. Start from the user/business trigger.
397. Trace identity and validation at each boundary.
398. Mark the ledger transaction boundary and commit confirmation.
399. Define event/projection side effects after commit.
400. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# Order Cancellation and Reservation Release
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The cancellation state machine exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.

- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 81 - Receipt Verification Flow

Verify that a local printed/PDF receipt has not been changed. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Canonicalize or store exact receipt bytes.

Implementation Objective

The concrete output of this chapter is: Hash verification workflow. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Canonicalize or store exact receipt bytes.
- Compute SHA-256 locally.
- Compare digest to on-chain receipt/order hash and display a clear pass/fail result.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

401. Start from the user/business trigger.
402. Trace identity and validation at each boundary.
403. Mark the ledger transaction boundary and commit confirmation.
404. Define event/projection side effects after commit.
405. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# Receipt Verification Flow
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The hash verification workflow exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.

- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 82 - Daily Reconciliation and Closing

Compare blockchain totals, PostgreSQL projections, cash drawer/terminal summaries, and stock movements. An end-to-end flow is the best way to validate architecture because it crosses identity, API, chaincode, ordering, commit, event processing, projection, and UI boundaries. Each step must have a defined source of truth, failure behavior, and retry strategy.

Design rule: Never rely on only one data source for reconciliation.

Implementation Objective

The concrete output of this chapter is: Closing checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Never rely on only one data source for reconciliation.
- Record discrepancies and manager resolution.
- Use immutable close-session records if formal daily sign-off is required.

Write a sequence diagram and a failure table side by side. For each step, record: component, input, output, source of truth, retry safety, timeout, and compensation. This prevents the common mistake of treating blockchain commit as equivalent to all downstream projections and notifications having succeeded.

Implementation Procedure

406. Start from the user/business trigger.
407. Trace identity and validation at each boundary.
408. Mark the ledger transaction boundary and commit confirmation.
409. Define event/projection side effects after commit.
410. Exercise rollback/retry/recovery for every failure point.

Implementation Example

```
# Daily Reconciliation and Closing
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The closing checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Dual writes are attempted to blockchain and SQL in the same logical step without an event/projection strategy.
- Recovery path cannot determine whether a transaction committed after an HTTP timeout.
- A later system overwrites historical values.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part IX - Security, Privacy, and Abuse Resistance

A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Chapter 83 - Threat Model for an Offline Blockchain Store

Analyze malicious staff, compromised endpoints, stolen keys, malware, database tampering, and hardware failure. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Offline does not mean safe from insiders or removable-media malware.

Implementation Objective

The concrete output of this chapter is: Threat register. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Offline does not mean safe from insiders or removable-media malware.
- Blockchain protects historical integrity only within the assumptions of key control and distributed administration.
- Endpoint compromise can submit valid-looking bad transactions if credentials are stolen.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

411. Identify asset and threat actor.
412. Apply least privilege at network, OS, API, and chaincode layers.
413. Protect keys/secrets and log privileged operations.
414. Test denial/revocation behavior.
415. Document incident containment and recovery.

Implementation Example

```
# Threat Model for an Offline Blockchain Store
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The threat register exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 84 - Network Segmentation and Firewall Policy

Limit which LAN components can talk to peers, orderers, CAs, databases, and management interfaces. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: POS clients talk to API, not peers/orderers.

Implementation Objective

The concrete output of this chapter is: Port matrix. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- POS clients talk to API, not peers/orderers.
- CA admin endpoints live on management segments.
- Default-deny between VLANs and explicitly allow required gRPC/HTTPS/database paths.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

416. Identify asset and threat actor.
417. Apply least privilege at network, OS, API, and chaincode layers.
418. Protect keys/secrets and log privileged operations.
419. Test denial/revocation behavior.
420. Document incident containment and recovery.

Implementation Example

```
# Network Segmentation and Firewall Policy
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The port matrix exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 85 - Private Key Protection

Protect Fabric enrollment keys and TLS keys from theft and accidental copying. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Use restrictive OS permissions and encrypted backups.

Implementation Objective

The concrete output of this chapter is: Key handling SOP. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use restrictive OS permissions and encrypted backups.
- Consider HSM/PKCS#11 for high-value production identities.
- Rotate compromised identities and revoke certificates promptly.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

421. Identify asset and threat actor.
422. Apply least privilege at network, OS, API, and chaincode layers.
423. Protect keys/secrets and log privileged operations.
424. Test denial/revocation behavior.
425. Document incident containment and recovery.

Implementation Example

```
# Private Key Protection
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The key handling sop exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 86 - Certificate Renewal and Revocation

Operate identity lifecycle without relying on public services. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Track expiration dates centrally.

Implementation Objective

The concrete output of this chapter is: Certificate lifecycle calendar. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Track expiration dates centrally.
- Test certificate rotation before emergency conditions.
- Distribute updated CRLs/trust material through controlled configuration processes.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

426. Identify asset and threat actor.
427. Apply least privilege at network, OS, API, and chaincode layers.
428. Protect keys/secrets and log privileged operations.
429. Test denial/revocation behavior.
430. Document incident containment and recovery.

Implementation Example

```
# Certificate Renewal and Revocation
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The certificate lifecycle calendar exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 87 - API Authorization and Least Privilege

Enforce business permissions consistently across UI, API, and chaincode. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Customer endpoints cannot call staff ledger methods.

Implementation Objective

The concrete output of this chapter is: Authorization matrix. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Customer endpoints cannot call staff ledger methods.
- Warehouse roles cannot issue financial refunds.
- Audit roles are read-only by design unless explicitly approved.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

431. Identify asset and threat actor.
432. Apply least privilege at network, OS, API, and chaincode layers.
433. Protect keys/secrets and log privileged operations.
434. Test denial/revocation behavior.
435. Document incident containment and recovery.

Implementation Example

```
# API Authorization and Least Privilege
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The authorization matrix exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 88 - PII Minimization and Pseudonymization

Keep customer-identifying data outside immutable blocks wherever possible. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Use customerRef hashes/opaque IDs in ledger records.

Implementation Objective

The concrete output of this chapter is: PII map. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use customerRef hashes/opaque IDs in ledger records.
- Store name/contact/address in PostgreSQL with retention controls.
- Never hash low-entropy secrets such as phone numbers without a keyed construction if anonymity is expected.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

436. Identify asset and threat actor.
437. Apply least privilege at network, OS, API, and chaincode layers.
438. Protect keys/secrets and log privileged operations.
439. Test denial/revocation behavior.
440. Document incident containment and recovery.

Implementation Example

On-chain order:

```
{
  "orderId": "ORD-2026-000123",
  "customerRef": "cust_93b7...", // opaque or keyed pseudonym
  "totalCents": 12900,
  "status": "COMPLETED"
}
```

Off-chain customer table:

customerRef -> name, phone, address, preferences, retention metadata

Verification Checklist

- The pii map exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.

- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 89 - Secure Offline Update Process

Patch servers and applications without silently reintroducing Internet dependencies or unverified binaries. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Acquire updates on a controlled staging station.

Implementation Objective

The concrete output of this chapter is: Update SOP. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Acquire updates on a controlled staging station.
- Verify vendor signatures/checksums and malware-scan media.
- Test in an offline staging environment before production rollout.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

441. Identify asset and threat actor.
442. Apply least privilege at network, OS, API, and chaincode layers.
443. Protect keys/secrets and log privileged operations.
444. Test denial/revocation behavior.
445. Document incident containment and recovery.

Implementation Example

```
# Secure Offline Update Process
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The update sop exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 90 - Incident Response and Kill Switches

Define how to stop writes, isolate terminals, revoke identities, and preserve evidence during an incident. A local-only network reduces exposure to remote Internet attacks but does not remove security requirements. Insider abuse, stolen credentials, malware, removable media, misconfiguration, lateral movement, and physical access remain important threats. Security must therefore be layered across network, OS, identity, API, chaincode, database, backups, and operations.

Design rule: Keep a documented emergency path to disable API write routes.

Implementation Objective

The concrete output of this chapter is: Incident runbook. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Keep a documented emergency path to disable API write routes.
- Revoke compromised certificates and rotate secrets.
- Preserve logs and ledger copies before recovery changes.

Apply defense in depth. A role hidden in the UI must also be denied by the API and, for critical ledger writes, by chaincode. A network firewall should block paths that application authorization does not need. Critical private keys should be inaccessible even to processes that only need public certificates.

Implementation Procedure

446. Identify asset and threat actor.
447. Apply least privilege at network, OS, API, and chaincode layers.
448. Protect keys/secrets and log privileged operations.
449. Test denial/revocation behavior.
450. Document incident containment and recovery.

Implementation Example

```
# Incident Response and Kill Switches
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The incident runbook exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Private keys are copied into shared folders.
- Firewall allows POS terminals direct access to peer/orderer admin interfaces.
- Sensitive customer/payment data is written to immutable blocks.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part X - Testing and Quality Engineering

Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Chapter 91 - Chaincode Unit Testing Strategy

Test pure domain rules and contract validation without depending exclusively on a live network. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: Extract total/quantity/state-transition calculations into pure functions.

Implementation Objective

The concrete output of this chapter is: Unit-test plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Extract total/quantity/state-transition calculations into pure functions.
- Cover boundaries and invalid transitions.
- Integration tests still validate actual Fabric behavior.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

451. Create deterministic seed data.
452. Run happy-path and boundary cases.
453. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
454. Reconcile ledger, projection, and business totals.
455. Automate repeatable tests and retain evidence.

Implementation Example

```
# Chaincode Unit Testing Strategy
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The unit-test plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.
- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 92 - Fabric Integration Test Environment

Run automated end-to-end tests against a disposable local network. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: Bring up test network, deploy chaincode, seed data, run API tests, tear down cleanly.

Implementation Objective

The concrete output of this chapter is: Integration harness. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Bring up test network, deploy chaincode, seed data, run API tests, tear down cleanly.
- Use fixed test identities and deterministic data.
- Never reuse production CA keys in test environments.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

456. Create deterministic seed data.
457. Run happy-path and boundary cases.
458. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
459. Reconcile ledger, projection, and business totals.
460. Automate repeatable tests and retain evidence.

Implementation Example

```
# Fabric Integration Test Environment
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The integration harness exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.

- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 93 - Checkout and Inventory Concurrency Tests

Prove stock cannot be oversold under concurrent submissions. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: Run parallel checkout submissions against low stock.

Implementation Objective

The concrete output of this chapter is: Concurrency suite. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Run parallel checkout submissions against low stock.
- Verify exactly allowed quantities commit.
- Assert losing transactions are surfaced to clients clearly.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

461. Create deterministic seed data.
462. Run happy-path and boundary cases.
463. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
464. Reconcile ledger, projection, and business totals.
465. Automate repeatable tests and retain evidence.

Implementation Example

```
# Checkout and Inventory Concurrency Tests
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The concurrency suite exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.

- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 94 - Negative and Security Test Cases

Test unauthorized roles, malformed payloads, replay/idempotency, oversized inputs, and forged references. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: Expect denial at API and/or chaincode layers.

Implementation Objective

The concrete output of this chapter is: Security test matrix. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Expect denial at API and/or chaincode layers.
- Test duplicate clientRequestId.
- Test a valid certificate from the wrong MSP.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

466. Create deterministic seed data.
467. Run happy-path and boundary cases.
468. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
469. Reconcile ledger, projection, and business totals.
470. Automate repeatable tests and retain evidence.

Implementation Example

```
# Negative and Security Test Cases
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The security test matrix exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.
- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 95 - Network Failure and Quorum Tests

Test behavior during peer loss, orderer loss, database loss, and temporary partitions. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: A three-node Raft orderer should continue with one node down while quorum remains.

Implementation Objective

The concrete output of this chapter is: Resilience test plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- A three-node Raft orderer should continue with one node down while quorum remains.
- Document which read/write paths degrade when specific peers fail.
- Validate recovery after reconnect.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

471. Create deterministic seed data.
472. Run happy-path and boundary cases.
473. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
474. Reconcile ledger, projection, and business totals.
475. Automate repeatable tests and retain evidence.

Implementation Example

TEST: Stop orderer2

EXPECT: three-node Raft cluster continues with orderer1 + orderer3.

TEST: Stop two orderers

EXPECT: new writes cannot achieve ordering quorum; reads from existing peer state may continue.

TEST: Restart orderers

EXPECT: cluster catches up; no duplicate checkout after API idempotent retry.

Verification Checklist

- The resilience test plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.
- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 96 - Backup and Restore Tests

Treat restore testing as mandatory, not optional documentation. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: Restore on isolated hardware.

Implementation Objective

The concrete output of this chapter is: Restore drill. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Restore on isolated hardware.
- Verify channel/ledger state, certificates, API database, projections, and receipt objects.
- Record measured RPO/RTO.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

476. Create deterministic seed data.
477. Run happy-path and boundary cases.
478. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
479. Reconcile ledger, projection, and business totals.
480. Automate repeatable tests and retain evidence.

Implementation Example

```
# Backup and Restore Tests
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The restore drill exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.

- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 97 - User Acceptance and Store Operations Testing

Test real cashier/warehouse workflows, not only API correctness. Blockchain systems need more than unit tests. Correctness depends on business rules, concurrent transactions, endorsement, ordering, commit validation, event delivery, identity permissions, and recovery after faults. Tests should intentionally create conflicts and failures instead of validating only the happy path.

Design rule: Measure scan-to-receipt time.

Implementation Objective

The concrete output of this chapter is: UAT scripts. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Measure scan-to-receipt time.
- Test operator mistakes and recovery screens.
- Verify audit and closing reports with store managers.

Tests should assert ledger state, transaction validity, emitted events, PostgreSQL projection state, and visible API outcome. A passing HTTP 200 alone is insufficient evidence. Capture transaction IDs so a failed test can be traced through peer/orderer/application logs.

Implementation Procedure

481. Create deterministic seed data.
482. Run happy-path and boundary cases.
483. Add concurrent, duplicate, unauthorized, malformed, and failure scenarios.
484. Reconcile ledger, projection, and business totals.
485. Automate repeatable tests and retain evidence.

Implementation Example

```
# User Acceptance and Store Operations Testing
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The uat scripts exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Only happy-path tests are executed.

- Concurrency/MVCC conflicts are treated as rare and ignored.
- Restore procedures exist on paper but have never been executed.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part XI - Operations, Backup, and Disaster Recovery

Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Chapter 98 - Daily Startup and Shutdown Procedures

Provide predictable service order and health verification for planned maintenance. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Start foundational services before application writes.

Implementation Objective

The concrete output of this chapter is: Operations checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Start foundational services before application writes.
- Use health checks rather than assuming container running means service healthy.
- Gracefully stop application writes before disruptive ledger maintenance.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

486. Define trigger, owner, prerequisites, and maintenance window.
487. Take/verify backups before risky changes.
488. Execute scripted steps with health checks between phases.
489. Validate ledger/application consistency after the change.
490. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
# Daily Startup and Shutdown Procedures
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The operations checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.

- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 99 - Health Monitoring and Local Alerting

Detect disk, process, certificate, peer, orderer, CouchDB, PostgreSQL, and projection failures. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Monitor peer/orderer process and ledger growth.

Implementation Objective

The concrete output of this chapter is: Health dashboard. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Monitor peer/orderer process and ledger growth.
- Alert on certificate expiration well in advance.
- Track event projection lag.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

491. Define trigger, owner, prerequisites, and maintenance window.
492. Take/verify backups before risky changes.
493. Execute scripted steps with health checks between phases.
494. Validate ledger/application consistency after the change.
495. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
# Health Monitoring and Local Alerting
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The health dashboard exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.

- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 100 - Backup Architecture

Create coordinated backups for Fabric ledger data, CA material, application database, configuration, and receipt/object storage. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Use offline/immutable backup copies separated from primary servers.

Implementation Objective

The concrete output of this chapter is: Backup plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use offline/immutable backup copies separated from primary servers.
- Encrypt backups containing private keys or PII.
- Document consistency rules for each component.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

496. Define trigger, owner, prerequisites, and maintenance window.
497. Take/verify backups before risky changes.
498. Execute scripted steps with health checks between phases.
499. Validate ledger/application consistency after the change.
500. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
backup-set/YYYY-MM-DD/
fabric/
  orderer-ledger/
  peer-ledger/
  channel-config/
  chaincode-packages/
identities/
  ca-config-encrypted/
  recovery-admin-encrypted/
app/
  postgres.dump
  receipts/
  config/
manifest/
  SHA256SUMS
  versions.txt
  restore-instructions.txt
```

Verification Checklist

- The backup plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.
- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 101 - Ledger Snapshots and Peer Recovery

Use supported ledger snapshot/rejoin mechanisms as part of large-ledger operations and recovery planning. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Understand the difference between peer filesystem backups and Fabric ledger snapshot capabilities.

Implementation Objective

The concrete output of this chapter is: Snapshot runbook. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Understand the difference between peer filesystem backups and Fabric ledger snapshot capabilities.
- Test channel join/recovery in a lab.
- Keep trusted channel configuration and identity material available.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

501. Define trigger, owner, prerequisites, and maintenance window.
502. Take/verify backups before risky changes.
503. Execute scripted steps with health checks between phases.
504. Validate ledger/application consistency after the change.
505. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
# Ledger Snapshots and Peer Recovery
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The snapshot runbook exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.
- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 102 - PostgreSQL Projection Rebuild

Recover the read model from authoritative ledger events/data after database loss. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Projection tables are disposable compared with the ledger if designed correctly.

Implementation Objective

The concrete output of this chapter is: Rebuild utility. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Projection tables are disposable compared with the ledger if designed correctly.
- Rebuild in deterministic order and checkpoint progress.
- Validate row counts/totals after rebuild.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

506. Define trigger, owner, prerequisites, and maintenance window.
507. Take/verify backups before risky changes.
508. Execute scripted steps with health checks between phases.
509. Validate ledger/application consistency after the change.
510. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
# PostgreSQL Projection Rebuild
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The rebuild utility exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.

- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 103 - Chaincode Upgrade and Migration

Upgrade smart-contract logic without losing or corrupting existing ledger state. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Increment sequence/version through Fabric chaincode lifecycle.

Implementation Objective

The concrete output of this chapter is: Upgrade runbook. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Increment sequence/version through Fabric chaincode lifecycle.
- Use backward-compatible state readers and explicit migrations if schema changes.
- Run regression and reconciliation tests before approval/commit.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

511. Define trigger, owner, prerequisites, and maintenance window.
512. Take/verify backups before risky changes.
513. Execute scripted steps with health checks between phases.
514. Validate ledger/application consistency after the change.
515. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
# Chaincode Upgrade and Migration
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The upgrade runbook exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.

- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 104 - Certificate Rotation Operations

Rotate application, peer, orderer, CA, and TLS certificates safely. Operational quality determines whether a blockchain system remains trustworthy after deployment. Certificates expire, disks fill, peers are replaced, software is patched, backups are restored, and chaincode evolves. Every such event needs a tested runbook that preserves availability and auditability.

Design rule: Plan overlapping trust roots/certs.

Implementation Objective

The concrete output of this chapter is: Rotation checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Plan overlapping trust roots/certs.
- Update connection profiles/configuration atomically where possible.
- Keep break-glass admin identity secured.

A runbook should be executable by someone other than its author. Include prechecks, exact commands, expected output, stop/rollback conditions, verification steps, and evidence to retain. Schedule periodic drills for restore and certificate rotation instead of waiting for an emergency.

Implementation Procedure

516. Define trigger, owner, prerequisites, and maintenance window.
517. Take/verify backups before risky changes.
518. Execute scripted steps with health checks between phases.
519. Validate ledger/application consistency after the change.
520. Record outcome, versions, certificates, and rollback status.

Implementation Example

```
# Certificate Rotation Operations
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The rotation checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Backups contain data but not the required keys/configuration to restore.

- Certificate renewal is started after expiration.
- Upgrade is performed without a rollback snapshot and compatibility test.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part XII - Performance, Scaling, and Advanced Designs

Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Chapter 105 - Performance Measurement for Checkout Workloads

Benchmark realistic order sizes, concurrent terminals, and event/projection load. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: Measure end-to-end commit latency, not only proposal time.

Implementation Objective

The concrete output of this chapter is: Benchmark plan. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Measure end-to-end commit latency, not only proposal time.
- Separate query throughput from write throughput.
- Include CouchDB/index and PostgreSQL projection costs.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

521. Write the problem that the advanced feature solves.
522. Measure the current system and define success criteria.
523. Prototype in a lab with production-like data.
524. Quantify added complexity and failure modes.
525. Adopt only if benefits exceed operational cost.

Implementation Example

```
# Performance Measurement for Checkout Workloads
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The benchmark plan exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.
- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 106 - Block Cutting and Ordering Tradeoffs

Understand how batching affects throughput and latency. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: Larger blocks can improve throughput but increase wait latency.

Implementation Objective

The concrete output of this chapter is: Tuning worksheet. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Larger blocks can improve throughput but increase wait latency.
- Retail checkout usually values predictable latency over maximum batch throughput.
- Tune only after measurement.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

526. Write the problem that the advanced feature solves.
527. Measure the current system and define success criteria.
528. Prototype in a lab with production-like data.
529. Quantify added complexity and failure modes.
530. Adopt only if benefits exceed operational cost.

Implementation Example

```
# Block Cutting and Ordering Tradeoffs
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The tuning worksheet exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.

- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 107 - Peer and CouchDB Resource Sizing

Plan CPU, RAM, disk IOPS, and retention based on projected transaction volume. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: World-state size and query patterns influence CouchDB resource needs.

Implementation Objective

The concrete output of this chapter is: Sizing worksheet. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- World-state size and query patterns influence CouchDB resource needs.
- Ledger blockchain storage grows monotonically.
- Leave operational headroom for compaction, backup, and reindexing.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

531. Write the problem that the advanced feature solves.
532. Measure the current system and define success criteria.
533. Prototype in a lab with production-like data.
534. Quantify added complexity and failure modes.
535. Adopt only if benefits exceed operational cost.

Implementation Example

```
# Peer and CouchDB Resource Sizing
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The sizing worksheet exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.

- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 108 - State-Based Endorsement for High-Risk Records

Require stronger organizational approval for selected inventory or financial states. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: Use default chaincode endorsement for routine operations.

Implementation Objective

The concrete output of this chapter is: Endorsement design. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use default chaincode endorsement for routine operations.
- Apply state-based endorsement selectively to high-value or cross-org assets.
- Avoid policies that make normal checkout unavailable unnecessarily.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

536. Write the problem that the advanced feature solves.
537. Measure the current system and define success criteria.
538. Prototype in a lab with production-like data.
539. Quantify added complexity and failure modes.
540. Adopt only if benefits exceed operational cost.

Implementation Example

```
# State-Based Endorsement for High-Risk Records
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The endorsement design exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.
- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 109 - Multiple Stores and Branches

Extend the design to multiple physical locations on a private WAN/LAN without public Internet. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: Use storeId in keys and authorization.

Implementation Objective

The concrete output of this chapter is: Multi-branch architecture. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use storeId in keys and authorization.
- Choose channel vs shared-channel/private-data boundaries based on governance, not convenience.
- Plan local survivability if inter-branch links fail.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

541. Write the problem that the advanced feature solves.
542. Measure the current system and define success criteria.
543. Prototype in a lab with production-like data.
544. Quantify added complexity and failure modes.
545. Adopt only if benefits exceed operational cost.

Implementation Example

```
# Multiple Stores and Branches
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The multi-branch architecture exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.

- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 110 - Supplier Organization Integration

Add a supplier org for purchase-order or goods-receipt verification. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: Only put cross-company facts on shared ledgers.

Implementation Objective

The concrete output of this chapter is: Supplier extension. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Only put cross-company facts on shared ledgers.
- Keep negotiated pricing private using PDC where appropriate.
- Do not expose customer PII to supplier peers.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

546. Write the problem that the advanced feature solves.
547. Measure the current system and define success criteria.
548. Prototype in a lab with production-like data.
549. Quantify added complexity and failure modes.
550. Adopt only if benefits exceed operational cost.

Implementation Example

```
# Supplier Organization Integration
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The supplier extension exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.

- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 111 - Upgrade Path to Fabric 3.x and SmartBFT

Understand when new capabilities justify migration from the LTS baseline. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: Fabric 3.x adds SmartBFT and other changes; migration is not required for a trusted single-enterprise LAN.

Implementation Objective

The concrete output of this chapter is: Upgrade decision memo. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Fabric 3.x adds SmartBFT and other changes; migration is not required for a trusted single-enterprise LAN.
- BFT is useful when orderer operators may be mutually distrustful or compromised.
- Treat consensus migration as an architecture project with dedicated testing.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

551. Write the problem that the advanced feature solves.
552. Measure the current system and define success criteria.
553. Prototype in a lab with production-like data.
554. Quantify added complexity and failure modes.
555. Adopt only if benefits exceed operational cost.

Implementation Example

```
# Upgrade Path to Fabric 3.x and SmartBFT
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The upgrade decision memo exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.
- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 112 - When to Remove Blockchain

Define the conditions under which a conventional database is the better design. Advanced design choices should be introduced only when measurement or governance requirements justify them. More peers, more channels, stronger endorsement, BFT ordering, and multi-organization layouts can improve certain trust properties, but also increase latency, administrative work, and failure modes.

Design rule: If one admin controls all nodes and no independent audit/trust boundary exists, blockchain may add cost without meaningful assurance.

Implementation Objective

The concrete output of this chapter is: Architecture review checklist. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- If one admin controls all nodes and no independent audit/trust boundary exists, blockchain may add cost without meaningful assurance.
- If throughput/query simplicity dominates and immutability is not required, use a database plus append-only audit log.
- Architects should be able to justify both adopting and rejecting blockchain.

Prefer the simplest topology that satisfies the trust and availability model. Every additional organization, peer, orderer, channel, collection, policy, and index increases configuration surface area. Complexity should be purchased with a measurable security, governance, availability, or performance benefit.

Implementation Procedure

556. Write the problem that the advanced feature solves.
557. Measure the current system and define success criteria.
558. Prototype in a lab with production-like data.
559. Quantify added complexity and failure modes.
560. Adopt only if benefits exceed operational cost.

Implementation Example

```
# When to Remove Blockchain
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The architecture review checklist exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Complex governance is introduced before a simple baseline is stable.
- Stricter endorsement reduces availability for routine retail operations.
- BFT is adopted without independent orderer operators or a malicious-node threat model.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Part XIII - Full Implementation Labs

This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Chapter 113 - Lab 1 - Build the Offline Installation Media

Create a staging bundle of Fabric binaries/images, Node packages, application source, database images/packages, checksums, and documentation. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Generate SHA-256SUMS.

Implementation Objective

The concrete output of this chapter is: Offline media directory. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Generate SHA-256SUMS.
- Export Docker images.
- Test installation with the WAN physically disconnected.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

561. Prepare a clean environment and snapshot it.
562. Implement the described milestone using scripts/configuration under version control.
563. Run the required verification commands/tests.
564. Capture logs, screenshots or test output as evidence.
565. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 1 - Build the Offline Installation Media
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The offline media directory exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 114 - Lab 2 - Create the Three-Organization Fabric Network

Bring up CAs, enroll identities, deploy three Raft orderers and organization peers, create the shopping channel, and verify membership. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Capture all CLI commands in scripts.

Implementation Objective

The concrete output of this chapter is: Running Fabric network. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Capture all CLI commands in scripts.
- Use persistent volumes.
- Verify peers can query channel info.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

566. Prepare a clean environment and snapshot it.
567. Implement the described milestone using scripts/configuration under version control.
568. Run the required verification commands/tests.
569. Capture logs, screenshots or test output as evidence.
570. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 2 - Create the Three-Organization Fabric Network
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The running fabric network exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 115 - Lab 3 - Deploy Shopping Chaincode

Package, install, approve, commit, and initialize the shopping chaincode. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Use the Fabric chaincode lifecycle.

Implementation Objective

The concrete output of this chapter is: Deployed chaincode. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use the Fabric chaincode lifecycle.
- Record package ID, sequence, endorsement policy, and approvals.
- Verify query and submit from each authorized org.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

571. Prepare a clean environment and snapshot it.
572. Implement the described milestone using scripts/configuration under version control.
573. Run the required verification commands/tests.
574. Capture logs, screenshots or test output as evidence.
575. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Illustrative lifecycle sequence (exact paths/addresses depend on your network)
peer lifecycle chaincode package shoppingcc.tar.gz \
--path ../chaincode --lang node --label shoppingcc_1

peer lifecycle chaincode install shoppingcc.tar.gz
peer lifecycle chaincode queryinstalled

peer lifecycle chaincode approveformyorg \
--channelID shopping --name shoppingcc --version 1.0 \
--package-id <PACKAGE_ID> --sequence 1 \
--orderer orderer1.local:7050 --tls --cafile <ORDERER_CA>

peer lifecycle chaincode commit \
--channelID shopping --name shoppingcc --version 1.0 --sequence 1 \
--peerAddresses peer0.shop.local:7051 --tlsRootCertFiles <SHOP_TLS_CA> \
--peerAddresses peer0.warehouse.local:8051 --tlsRootCertFiles <WAREHOUSE_TLS_CA> \
--orderer orderer1.local:7050 --tls --cafile <ORDERER_CA>
```

Verification Checklist

- The deployed chaincode exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 116 - Lab 4 - Implement Product and Inventory Transactions

Implement product creation, stock receipt, stock adjustment, availability, and movement history. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Add validation and authorization.

Implementation Objective

The concrete output of this chapter is: Working chaincode module. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Add validation and authorization.
- Emit events.
- Write integration tests.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

576. Prepare a clean environment and snapshot it.
577. Implement the described milestone using scripts/configuration under version control.
578. Run the required verification commands/tests.
579. Capture logs, screenshots or test output as evidence.
580. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 4 - Implement Product and Inventory Transactions
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The working chaincode module exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 117 - Lab 5 - Implement Checkout and Idempotency

Create a single transaction that validates products, checks inventory, creates order state, records movements, and stores an idempotency marker. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Use integer money.

Implementation Objective

The concrete output of this chapter is: Checkout transaction. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use integer money.
- Use clientId.
- Test duplicate and concurrent submissions.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

581. Prepare a clean environment and snapshot it.
582. Implement the described milestone using scripts/configuration under version control.
583. Run the required verification commands/tests.
584. Capture logs, screenshots or test output as evidence.
585. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 5 - Implement Checkout and Idempotency
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The checkout transaction exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 118 - Lab 6 - Build the Fabric Gateway Backend

Connect Node.js API to the peer gateway and expose health, product, inventory, and checkout endpoints. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Use TLS.

Implementation Objective

The concrete output of this chapter is: Local REST API. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Use TLS.
- Set commit deadlines.
- Return correlation ID and txId.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

586. Prepare a clean environment and snapshot it.
587. Implement the described milestone using scripts/configuration under version control.
588. Run the required verification commands/tests.
589. Capture logs, screenshots or test output as evidence.
590. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 6 - Build the Fabric Gateway Backend
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The local rest api exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.

- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 119 - Lab 7 - Build the Event Projection Worker

Consume chaincode events and populate PostgreSQL read tables. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Persist checkpoints.

Implementation Objective

The concrete output of this chapter is: Projection worker. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Persist checkpoints.
- Handle replay safely.
- Rebuild from zero in test.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

591. Prepare a clean environment and snapshot it.
592. Implement the described milestone using scripts/configuration under version control.
593. Run the required verification commands/tests.
594. Capture logs, screenshots or test output as evidence.
595. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 7 - Build the Event Projection Worker
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The projection worker exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.

- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 120 - Lab 8 - Build the Local POS Web App

Create login, catalog, cart, checkout, pending/committed status, and receipt screens using only local assets. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: No CDN calls.

Implementation Objective

The concrete output of this chapter is: POS application. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- No CDN calls.
- Show network/health status.
- Prevent accidental duplicate submits.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

596. Prepare a clean environment and snapshot it.
597. Implement the described milestone using scripts/configuration under version control.
598. Run the required verification commands/tests.
599. Capture logs, screenshots or test output as evidence.
600. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 8 - Build the Local POS Web App
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The pos application exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.

- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 121 - Lab 9 - Implement Returns and Refund Approvals

Add return validation, manager approval rules, refund records, and inventory disposition. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Test partial returns.

Implementation Objective

The concrete output of this chapter is: Return workflow. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Test partial returns.
- Test repeated return attempts.
- Audit every approval.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

601. Prepare a clean environment and snapshot it.
602. Implement the described milestone using scripts/configuration under version control.
603. Run the required verification commands/tests.
604. Capture logs, screenshots or test output as evidence.
605. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 9 - Implement Returns and Refund Approvals
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The return workflow exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 122 - Lab 10 - Implement Receipt Hash Verification

Generate receipt PDF/text, hash exact bytes, anchor the digest, and build a verification screen. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Define canonical receipt format.

Implementation Objective

The concrete output of this chapter is: Receipt verification feature. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Define canonical receipt format.
- Store receipt file locally.
- Verify modified receipt fails.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

606. Prepare a clean environment and snapshot it.
607. Implement the described milestone using scripts/configuration under version control.
608. Run the required verification commands/tests.
609. Capture logs, screenshots or test output as evidence.
610. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 10 - Implement Receipt Hash Verification
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The receipt verification feature exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.

- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 123 - Lab 11 - Secure the LAN and Identities

Apply VLAN/firewall rules, file permissions, password policy, certificate expiry alerts, and backup encryption. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Run port scan from POS VLAN.

Implementation Objective

The concrete output of this chapter is: Security baseline. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Run port scan from POS VLAN.
- Confirm peer/orderer admin ports are unreachable.
- Test revoked user behavior.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

611. Prepare a clean environment and snapshot it.
612. Implement the described milestone using scripts/configuration under version control.
613. Run the required verification commands/tests.
614. Capture logs, screenshots or test output as evidence.
615. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 11 - Secure the LAN and Identities
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The security baseline exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Chapter 124 - Lab 12 - Run Failure, Backup, and Recovery Drills

Stop one orderer, stop a peer, interrupt projection DB, restore from backup, and document observed behavior. This lab converts the architecture into a verifiable implementation milestone. Work on a disposable lab environment first, record exact commands/configuration, and require an objective completion test before moving to the next lab. The target production network remains offline during normal operation.

Design rule: Measure outage and recovery time.

Implementation Objective

The concrete output of this chapter is: DR test report. The implementation should be reviewable by another engineer or operator without relying on undocumented assumptions. In an offline network, reproducibility is especially important because missing packages, public services, or implicit cloud dependencies cannot be fixed by downloading something during an outage.

Key Engineering Decisions

- Measure outage and recovery time.
- Validate no duplicate orders after client retry.
- Reconcile ledger and projection after recovery.

Use a lab evidence folder containing terminal transcripts, configuration hashes, test results, and a short report. A lab is complete only when it can be repeated from a clean environment using the written steps and produces the same functional verification result.

Implementation Procedure

616. Prepare a clean environment and snapshot it.
617. Implement the described milestone using scripts/configuration under version control.
618. Run the required verification commands/tests.
619. Capture logs, screenshots or test output as evidence.
620. Document issues and reset/repeat until the lab is reproducible.

Implementation Example

```
# Lab 12 - Run Failure, Backup, and Recovery Drills
INPUT -> validate identity and request
STATE -> read the minimum authoritative state
RULES -> enforce deterministic business/security constraints
WRITE -> commit one atomic business update where required
EVENT -> emit compact identifiers for projections/workflows
VERIFY -> query/inspect the committed result and retain txId evidence
```

Verification Checklist

- The dr test report exists in version-controlled documentation or source.
- The implementation works with the WAN/Internet disabled.
- Authorization and invalid-input behavior have explicit tests.
- The source of truth is documented for every value used by this component.
- A failure/retry path is defined and does not create duplicate business actions.

Common Failure Modes

- Commands work once interactively but are not scripted.
- Verification relies on container status instead of functional transactions.
- Lab uses production secrets or identities.

Expert Review Questions

- What can go wrong if this component succeeds but the next component fails?
- Which identity signs or authorizes the operation, and how is that identity revoked?
- Can the operation be safely retried after a timeout?
- Which data is immutable, which is mutable, and which can be deleted?
- How would an auditor prove the final business outcome from ledger and off-chain evidence?

Appendix A - Recommended Repository Structure

A full directory tree for infrastructure, chaincode, backend, frontend, database, scripts, tests, and operations documents.

```
shopping-blockchain/  
  infra/  
    fabric/  
      compose/  
      configtx/  
      crypto/  
      scripts/  
    db/  
    monitoring/  
  chaincode/  
    src/contracts/  
    src/models/  
    src/lib/  
    test/  
  backend/  
    src/fabric/  
    src/routes/  
    src/auth/  
    src/projection/  
    migrations/  
  frontend/  
    src/  
    public/  
  operations/  
    backup/  
    restore/  
    rotate-certs/  
    upgrade/  
  tests/  
    integration/  
    concurrency/  
    security/  
  docs/  
    architecture/  
    api/  
    runbooks/
```

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.

- A second operator/engineer reviewed the final production version.

Appendix B - Core TypeScript Chaincode Listing

A consolidated reference implementation of data models, authorization helpers, product/inventory/order/payment/return transactions, idempotency, and events.

```
// Consolidated pattern - abbreviated enough to remain readable in print.
// Production source should split models/helpers/contracts into separate files.

import { Context, Contract, Info, Transaction, Returns } from 'fabric-contract-api';

type Money = number; // integer cents only

type Inventory = {
  docType: 'inventory'; storeId: string; productId: string;
  onHand: number; reserved: number; updatedTx: string;
};

type Order = {
  docType: 'order'; orderId: string; storeId: string;
  status: 'COMPLETED' | 'CANCELLED' | 'PARTIALLY_RETURNED' | 'RETURNED';
  totalCents: Money; paymentStatus: string; customerRef?: string;
  createdTx: string;
};

@Info({title:'ShoppingContract', description:'Offline LAN shopping ledger contract'})
export class ShoppingContract extends Contract {
  private async getJson<T>(ctx: Context, key: string): Promise<T | undefined> {
    const b = await ctx.stub.getState(key);
    return b.length ? JSON.parse(b.toString('utf8')) as T : undefined;
  }
  private async putJson(ctx: Context, key: string, value: unknown): Promise<void> {
    await ctx.stub.putState(key, Buffer.from(JSON.stringify(value)));
  }
  private requireRole(ctx: Context, roles: string[]): void {
    const role = ctx.clientIdentity.getAttributeValue('role');
    if (!role || !roles.includes(role)) throw new Error('FORBIDDEN');
  }
  private assertInt(name: string, n: number, min=0): void {
    if (!Number.isSafeInteger(n) || n < min) throw new Error('INVALID_${name.toUpperCase()}');
  }

  @Transaction()
  public async receiveStock(ctx: Context, requestId: string, storeId: string, productId: string, qtyText: string): Promise<string> {
    this.requireRole(ctx, ['warehouse', 'manager']);
    const reqKey = `REQUEST~${requestId}`;
    const oldReq = await ctx.stub.getState(reqKey);
    if (oldReq.length) return oldReq.toString('utf8');

    const qty = Number(qtyText); this.assertInt('qty', qty, 1);
    const key = `INV~${storeId}~${productId}`;
    const inv = (await this.getJson<Inventory>(ctx, key)) ??
    {docType:'inventory',storeId,productId,onHand:0,reserved:0,updatedTx:''};
    inv.onHand += qty; inv.updatedTx = ctx.stub.getTxID();
    await this.putJson(ctx, key, inv);

    const movementId = ctx.stub.getTxID();
```

```

await this.putJson(ctx, `MOVE~${storeId}~${productId}~${movementId}`,
  {docType:'movement',movementId,storeId,productId,type:'RECEIPT',qty,txId:movementId});
const result = JSON.stringify({movementId,onHand:inv.onHand});
await ctx.stub.putState(reqKey,Buffer.from(result));
ctx.stub.setEvent('InventoryChanged',Buffer.from(JSON.stringify({storeId,productId,movementId})));
return result;
}
}

```

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix C - Fabric Gateway Backend Listing

A consolidated Node.js/TypeScript connection helper, transaction wrapper, Express routes, error mapping, and commit handling.

```
// Transaction wrapper used by REST commands
export async function submitJson(
  fn: string,
  args: string[],
): Promise<{txResult: any}> {
  try {
    const bytes = await contract.submitTransaction(fn, ...args);
    return { txResult: JSON.parse(Buffer.from(bytes).toString('utf8')) || '{}' };
  } catch (err: any) {
    // Map known deterministic chaincode messages to stable application codes.
    throw mapFabricError(err);
  }
}

// Query wrapper used when the ledger itself must be queried.
export async function evaluateJson(fn: string, ...args: string[]) {
  const bytes = await contract.evaluateTransaction(fn, ...args);
  return JSON.parse(Buffer.from(bytes).toString('utf8')) || '{}';
}
```

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix D - Docker Compose and Configuration Templates

Representative offline deployment templates for CAs, orderers, peers, CouchDB, PostgreSQL, API, and monitoring.

```
services:
  orderer1:
    image: hyperledger/fabric-orderer:2.5.16
    pull_policy: never
    environment:
      - ORDERER_GENERAL_LISTENADDRESS=0.0.0.0
      - ORDERER_GENERAL_LOCALMSPID=OrdererMSP
      - ORDERER_GENERAL_TLS_ENABLED=true
    volumes:
      - orderer1_data:/var/hyperledger/production/orderer
      - ./crypto/orderer1/msp:/var/hyperledger/orderer/msp:ro
      - ./crypto/orderer1/tls:/var/hyperledger/orderer/tls:ro

  peer0-shop:
    image: hyperledger/fabric-peer:2.5.16
    pull_policy: never
    environment:
      - CORE_PEER_ID=peer0-shop
      - CORE_PEER_LOCALMSPID=ShopMSP
      - CORE_PEER_TLS_ENABLED=true
      - CORE_LEDGER_STATE_STATEDATABASE=CouchDB
      - CORE_LEDGER_STATE_COUCHDBCONFIG_COUCHDBADDRESS=couchdb-shop:5984
    volumes:
      - peer0_shop_data:/var/hyperledger/production
      - ./crypto/shop/peer0/msp:/etc/hyperledger/fabric/msp:ro
      - ./crypto/shop/peer0/tls:/etc/hyperledger/fabric/tls:ro
```

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix E - REST API Specification

Endpoint list, request/response examples, error codes, idempotency headers, and role requirements.

```
GET /api/health
POST /api/auth/login
GET /api/products?q=&page=
POST /api/admin/products
GET /api/inventory/:storeId:productId
POST /api/warehouse/receipts
POST /api/orders/checkout      Idempotency-Key: <uuid>
GET /api/orders/:orderId
POST /api/orders/:orderId/returns Idempotency-Key: <uuid>
POST /api/refunds/:refundId/approve
GET /api/audit/transactions/:txId
POST /api/receipts/:orderId/verify
```

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix F - PostgreSQL Projection Schema

DDL examples for products, inventory, orders, lines, payments, movements, receipts, users, event checkpoints, and audit correlation.

```
CREATE TABLE event_checkpoint (  
  consumer_name text PRIMARY KEY,  
  next_block_no bigint NOT NULL  
);  
  
CREATE TABLE inventory_projection (  
  store_id text NOT NULL,  
  product_id text NOT NULL,  
  on_hand bigint NOT NULL,  
  reserved bigint NOT NULL,  
  available bigint NOT NULL,  
  ledger_tx_id text NOT NULL,  
  PRIMARY KEY (store_id, product_id)  
);
```

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix G - Test Matrix

A comprehensive acceptance matrix for functional, concurrency, security, failure, restore, and performance testing.

Item	Implementation Guidance
Functional	Create product, receive stock, checkout, cancel, return, refund, receipt verify.
Authorization	Customer/cashier/manager/warehouse/auditor role allow/deny cases.
Idempotency	Repeat same request key after success, timeout, and client reconnect.
Concurrency	Parallel sale of last stock; simultaneous receipt/adjustment/sale.
Faults	Peer down, one orderer down, PostgreSQL down, projection worker down, API restart.
Security	Wrong MSP, revoked cert, malformed JSON, large strings, path/body tampering.
Recovery	Restore database, restore peer/orderer in lab, rebuild projection, rotate certs.
Performance	Target terminal concurrency, order size, query volume, daily closing report.

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix H - Production Readiness Checklist

A go-live checklist spanning network, identities, backups, security, monitoring, business operations, and rollback.

- All production hosts have static addressing, local DNS, synchronized local time, patched OS, and documented owners.
- No UI or runtime service depends on a public CDN, public DNS, cloud auth, public package registry, or remote telemetry endpoint.
- Fabric peer/orderer/CA versions and container digests are pinned and recorded.
- TLS is enabled for all Fabric and API communications; trust roots are backed up and expiry monitored.
- Three-node Raft quorum failure test has passed.
- Chaincode lifecycle package, approvals, sequence, endorsement, and source commit are recorded.
- Checkout is idempotent and tested under concurrent last-item conditions.
- Customer PII/payment secrets are not stored on-chain.
- Backups have been restored successfully on isolated hardware.
- Projection database can be rebuilt or reconciled from ledger evidence.
- Security/firewall tests prove POS clients cannot access peer/orderer admin interfaces.
- Incident response includes write-disable, cert revocation, evidence preservation, and recovery steps.

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix I - Offline Maintenance Runbook

How to patch, rotate certificates, update chaincode, update API/UI, import approved images/packages, and verify system integrity without Internet.

1. Acquire approved update bundle on staging workstation.
2. Verify vendor signature/checksum and internal SHA-256 manifest.
3. Malware scan removable media and write-protect where possible.
4. Import into offline staging LAN, not production first.
5. Back up production and capture current versions/configuration.
6. Apply update in dependency order with health checks.
7. Execute regression, transaction, reconciliation, and restore tests.
8. Record versions, hashes, operator, date, and rollback outcome.

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Appendix J - Glossary and Official References

Definitions plus official Hyperledger Fabric documentation links used as the technical baseline.

Core Glossary

Item	Implementation Guidance
Peer	Fabric node that maintains ledger state and executes/hosts chaincode.
Orderer	Node in the ordering service that establishes final transaction order and blocks.
MSP	Membership Service Provider configuration that defines trusted identities for an organization.
Chaincode	Fabric smart-contract program implementing agreed business logic.
World state	Current key/value facts for a channel/chaincode namespace.
Blockchain	Immutable ordered transaction/block history that produced current state.
Endorsement	Execution/signature requirements that a transaction proposal must satisfy.
Fabric Gateway	Peer service and client API that simplifies transaction evaluation, endorsement, submission, commit status, and events.
PDC	Private Data Collection, allowing selected channel organizations to hold plaintext private data while hashes provide ledger evidence.
MVCC	Version-based validation that detects stale/concurrent reads during commit validation.

Official Technical References

- Hyperledger Fabric docs - <https://hyperledger-fabric.readthedocs.io/en/latest/>
- Fabric installation - <https://hyperledger-fabric.readthedocs.io/en/latest/install.html>
- Production deployment overview - https://hyperledger-fabric.readthedocs.io/en/latest/deployment_guide_overview.html
- Fabric Gateway - <https://hyperledger-fabric.readthedocs.io/en/latest/gateway.html>
- Ledger concepts - <https://hyperledger-fabric.readthedocs.io/en/latest/ledger/ledger.html>
- Ordering service - https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html
- Chaincode lifecycle - https://hyperledger-fabric.readthedocs.io/en/latest/chaincode_lifecycle.html
- Writing chaincode - <https://hyperledger-fabric.readthedocs.io/en/latest/chaincode4ade.html>
- CouchDB state database - https://hyperledger-fabric.readthedocs.io/en/latest/couchdb_as_state_database.html
- Private data - <https://hyperledger-fabric.readthedocs.io/en/latest/private-data-arch.html>
- Peer events - https://hyperledger-fabric.readthedocs.io/en/latest/peer_event_services.html
- Certificate management - https://hyperledger-fabric.readthedocs.io/en/latest/certs_management.html

- Fabric releases - <https://github.com/hyperledger/fabric/releases>
- Fabric Gateway Node client - <https://github.com/hyperledger/fabric-gateway>

Implementation Notes

Treat this appendix as a starting reference, not a substitute for environment-specific review. Replace example hostnames, certificates, secrets, IDs, and version pins with values from the controlled production inventory. Keep the final files in version control, export a signed/checksummed release bundle, and test it on a clean offline staging environment before use in production.

Validation Checklist

- All examples have been adapted to the real organization names, internal DNS names, and ports.
- No placeholder private keys, passwords, or test secrets remain.
- Version pins match the approved offline artifact manifest.
- Recovery and rollback steps are documented.
- A second operator/engineer reviewed the final production version.

Final Implementation Roadmap - 12 Weeks

Item	Implementation Guidance
Week 1	Requirements, trust model, on-chain/off-chain matrix, threat model, hardware/network inventory.
Week 2	Offline artifact bundle, lab servers, local DNS/time, Docker/runtime, security baseline.
Week 3	Fabric CAs, organizations, identities, MSPs, TLS, three-node Raft orderers.
Week 4	Peers, CouchDB, shopping channel, lifecycle scripts, network health/failure tests.
Week 5	Product/inventory chaincode, authorization, idempotency, indexes, integration tests.
Week 6	Order/payment/return chaincode, concurrency tests, events, chaincode upgrade workflow.
Week 7	Fabric Gateway backend, auth, REST commands/queries, stable error model.
Week 8	Projection worker, PostgreSQL read models, receipt/document storage and hashing.
Week 9	POS/customer/warehouse/audit web UI and offline asset packaging.
Week 10	End-to-end flows, UAT, negative/security tests, performance measurements.
Week 11	Backup/restore drill, certificate rotation drill, incident response, monitoring/alerts.
Week 12	Production rehearsal, go-live checklist, rollback plan, operator training, controlled launch.

Definition of done: The system is not complete when a checkout demo succeeds. It is complete when the WAN can be disconnected, authorized users can perform all supported business workflows, unauthorized actions are denied, failures/retries do not duplicate transactions, ledger and projections reconcile, backups can be restored, certificates can be rotated, and another operator can run the system using documented procedures.